University of Dhaka

Integrated Threat Detection Framework Using YARA and Network Anomaly Detection with Wazuh Dashboard for SOC Operations

Submitted By:

Exam Roll: 512

Registration No: H-125

Exam Roll: 519

Registration No: H-153

Submitted To the
Department of Computer Science and Engineering
University of Dhaka, Bangladesh
in partial fulfillment of the requirements for the
degree of Professional Masters in Information
and Cyber Security (PMICS)

Submission Date: 25th August, 2025

UNIVERSITY OF DHAKA

INTEGRATED THREAT DETECTION FRAMEWORK USING YARA AND NETWORK ANOMALY DETECTION WITH WAZUH DASHBOARD FOR SOC OPERATIONS

SUBMITTED BY
EXAM ROLL: 512
REGISTRATION NO: H-125
EXAM ROLL: 519
REGISTRATION NO: H-153

SUBMITTED TO THE
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
UNIVERSITY OF DHAKA, BANGLADESH
IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE
DEGREE OF PROFESSIONAL MASTERS IN INFORMATION
AND CYBER SECURITY (PMICS)

SUBMISSION DATE: 25TH AUGUST, 2025

Abstract

Our study challenges traditional cybersecurity paradigms by introducing an integrated and adaptive threat detection framework that addresses the limitations of static signature-based systems. We developed a distributed multi-node Wazuh infrastructure, leveraging its core components to establish a robust Security Operation Center (SOC) platform. The framework's key innovation is its fusion of YARA-based file scanning with unsu- pervised machine learning models, specifically Isolation Forest and One-Class SVM, to provide a multi-layered defense against both known and unknown threats. During attack simulations, our system successfully detected malicious files and validated the attacks by plotting clear network anomalies, confirming its efficacy against modern threats. While we encountered challenges such as hardware limitations that restricted the scope of our simulations, this project successfully demonstrates a highly effective, real-time, and adap- tive defense solution that significantly advances threat detection capabilities.

Contents

List of Figures

Chapter 1

Introduction

The present-day digital environment is characterized by an unprecedented degree of interconnectedness, which, in addition to allowing international communication and trade, has created an enormous attack surface to growingly sophisticated cyber threats. The threat environment is no longer simple viruses, but has now become diverse, with polymorphic malware, stealth zero-day exploits, and much more organized state-sponsored threats. In such a landscape, signature-based monolithic security models have been demonstrating to be grossly inadequate. Such traditional mechanisms merely screened through an attack that had already been identified, and thus are not only fundamentally unable to detect and respond to new and unseen attack vectors, but also to subtle behavioral anomalies that fail to follow a predefined rule. The fact that many security tools are siloed only compounds the problem, introducing key blind spots and lengthening incident response in Security Operations Centers (SOCs).

To fix this irrevocable security discrepancy, we must be willing to change our way of thinking- with a shift in paradigm- toward proactive, behaviorally oriented analytics. The proposal of this masters is to develop a unified and intelligent model of threat detection to achieve a comprehensive state of security through the synergic fusion of various technologies which are designed to be advanced. We find our line of research in the intersection of exactness of YARA rules, which can be deemed as a powerful tool in detecting file-based malicious payload, and evaporation of unsupervised machine learning models. In particular, we use Isolation Forest and One-Class SVM to process huge streams of network traffic data in realtime and detect aberrant network traffic indicative of a compromise.

This tiered protection solution is built on a solid distributed multi-node platform Wazuh. Wazuh, a well-known Open-Source Security Information and Event Management (SIEM) tool, is the backbone of the collection, correlation and a centralized visualization dashboard of security events. The combination of the outputs of our YARA and machine learning components into the Wazuh SIEM enables our SOC analysts to receive a singular, unified picture of known and unknown threats. This collective solution also improves the process of detection as well as the ease of investigation in the process itself. The project can be regarded as a proof-of-concept of a more intelligent, resilient and holistic cybersecurity framework and fills a much needed research vacuum and potentially provides a solution to the ever-growing threat landscape as it pertains to digital assets.

1.1 Problem Statement

The network traffic pattern and downloaded files are detection functions done in isolation in the present cybersecurity systems and fails to provide a comprehensive view of the threats. Security tools, such as YARA, follow a method of static signature detection that does not work when malware mutates and conventional network monitoring tools present low adaptability in recognizing new malware. Automated, coupled with machine learning anomaly detection, capability of YARA scanning is an area with untapped potential, particularly with Wazuh and other such SIEM systems. When these functions exist independently, SOC environments can be characterized by delayed threat detection and missed incidents, which underscores the need to have an integrated real-time threat detection solution in place.

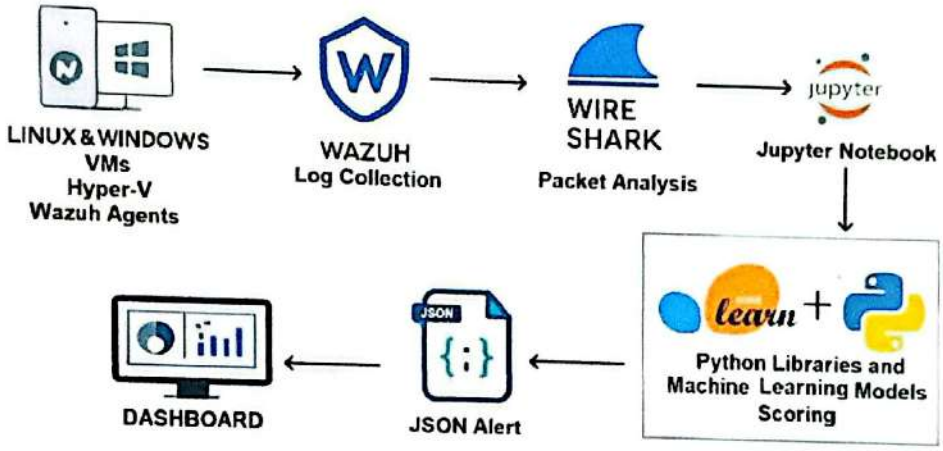

Figure 1.1: Our solution to the problem statement.

1.2 Research Gap

Very few open-source frameworks integrate YARA and unsupervised machine learn- ing as Wazuh does. Creation of powerful anomaly detecting algorithms will, in the short-term, necessitate the availability of vast real-time datasets that are presently not available. The literature does not present any research studies on how to develop SOC-ready visual integrations interlinking hybrid detection systems.

1.3 Objectives and Outcomes

The proposed project will implement a threat detection solution to combine machine learning-based network anomaly detection and YARA-based malware scanning into the Wazuh SIEM dashboard visualisations. Its three core aims are the development of custom YARA rules to detect threats to files, and development of unsupervised ML models composed of the Isolation forest and One- Class SVM algorithms to detect anomalies, together with their integration into Wazuh to generate alerts in real-time and present them in a dashboard. By the needs of safety and reproducibility, the system is to work in individual tests performed with Windows-based and Linux-based virtual machines.

1.4 Possible Outcomes

This project will provide three major results: a workable prototype that detects known malware and unidentified network anomalies and instantly displays the alerts on the Wazuh dashboard operations or operations in parallel offer modular capabilities on which the SOC can rely on. The project manages to develop reusable YARA rules and trained machine learning models along with the production of detailed documentation as evidence of the success of the project.

1.5 Project Organization

The project outline is as follows: Chapter 2, Liter- ature Review presents the current state of research YARA, Malware Classification, Machine Learning, Network Anomaly Detection, Behavioral Detection, and WazuhSIEM (Security Information and Event Management). Chapter 3 entails Methodologies and the specifications of the system architecture and integration strategies and gives details on Isolation Forest models. Chapter 4 deals with evaluation and testing results of the entire activity of the SOC. Chapter 5 is the data recording/reporting of each detail of the malware. It is also a summary of the research contribution and highlights the main findings, and includes possible enhancements to the future of the SOCs. In the final chapter, Chapter 6, conclusions are provided to this work on the project.

Chapter 2

Literature Review

Automation of YARA rules generation in order to enhance precision and scalability has also been studied in recent times. PackGenome suggested a gene-like methodology that detects new rules on the basis of packer-specific byte sequences in unpacking scripts and systematically creates new YARA rules. The method also substantially minimizes false positive cases, and zeroes out false negatives within large collections, compared to both existing rule sets and tools [7]. Although PackGenome aims mainly at improving the quality and automation of rule generation, our project aims at the entire process of bringing manually designed YARA rules into operation in a Wazuh-based SOC. Through the Wazuh dashboard, our framework allows real-time alerting and visualization via the combination of these rules with unsupervised anomaly detection models including Isolation Forest and One-Class SVM. Collectively, these supplementary instructions emphasize the state of sophistication of automated rule generation to combat malware and the need to incorporate hybrid detection measures in a SOC setup.

A further important development in the field of automated signature generation is NeuroYara, that uses deep language modeling and discriminatative n-gram encoding to automate the generation of high-quality YARA rules. Compared to conventional systems based on extensive precomputed tables of n-grams, NeuroYara is distinct in its use of learning-to-rank neural nets for determining good correlations among n-grams, and its use of hierarchical clustering to compactly represent the set of meaningful rules in the outputs. Such a framework minimises the reliance on manual analyst experience, and provides lower false positive rates and beats not only currently available automated tools, but also hand-written rules [9]. Whereas NeuroYara aims to enhance the performance and scalability of constructing YARA rules, we concentrate on the practical implementation of YARA in a Wazuh-based SOC when custom rules are combined with machine learning-based anomaly detection (Isolation Forest and One-Class SVM). Due to the combination of these works, these two directions in malware detection are complementary: on the one hand, NeuroYara focuses on automatizing and making smart rules, and our work on the other hand is connected to how to integrate and operate the same rules in an actual SOC and SIEM settings.

Rule generation techniques were included to develop automated biclustering methods that developed effective rules that sampled several classes of malware [12]. The APIARY was characterized by an automated YARA rule generation process that used ob- served sequences of API calls extracted as binary code. We have placed behavioral detection capabilities into rules themselves and this not only advanced their accuracy, but made them more capable of overcoming interference.

Nefariouse use of web pages in an effort to confuse those who are supposed to contact them attempting to obscure their identities [14]. The integration of behavioral detection indicators to automatically generate the rules helps YARA in its ability to fight malware in the current threat detection systems.

YARA allows the users to build robust rules to identify and classify malware by identifying behavioral techniques, patterns in text, and binary patterns. The effectiveness of a Static YARA rule needs to be tuned when the malware programs develop higher sophistication In a recent research paper, an innovative procedure of obtaining YARA rules features, as part of the rule-based detection mechanism to be classified as malware, is described, in which we gain extensive insight into the capabilities of the machine learning technique to enhance rule-based malware detection systems [4].

Intrusion detection systems take on both signature-based and anomaly-based systems. The signature-based approach (i.e. YARA rules) is useful to detect known malware, whereas anomaly detection tools (i.e. Isolation Forest and Extended Isolation Forest (EIF)) identify abnormal behavior in network or host traffic. Other recent work that applies EIF to Zeek logs has shown that tuning hyperparameters can greatly enhance detection success in high dimensional network data [11].In our own project, we use this type of hybrid detector by pairing signature-based detection via custom YARA rules and anomaly detection to identify threats proactively. Wazuh is highly extensible, and we can add custom decoders, create interactive dashboards available in real-time and send active responses when suspicious activity is identified. Further, integrating vulnerability check (such as VirusTotal scans) will help the system to recognize and respond to malware practices/ activities on both Linux and Windows agents. The combination of these measures facilitates an effective real-time monitoring schema directly corresponding to the methodologies demonstrated effective during recent researches and valuable in terms of practical deployment in enterprise set-ups.

File-level analysis is more effective when it is integrated with network anomaly tracking to identify anomalous trends in data transmission such as data ex-filtration alongside botnet activity and movement laterally. Comparison of various elements of machine learning (supervised and unsupervised) in the detection of network traffic abnormalities give an adequate insight to this field. The study proves that Isolation Forest and One-Class SVM are also worthy approaches to detect new and soft anomalies [13]. Limitations and weaknesses According to researchers, the implementation of Explainable AI (XAI) methods resulted in an increase in detection accuracy through using techniques of a class-balancing system as well as visualization of feature importance, whereas the accuracy level of the ensemble learning model without the extra methods of elevating the detection level remained inferior to the one with the implementation of XAI techniques [5].

The paper, Overcoming the Limitations of the Traditional Rule-based SIEM Systems with Machine Learning Techniques, focuses on the improvement of threat detection in Wazuh by overcoming several limitations of rule-based approaches to SIEMs, including limited adaptability to new threats, and an excessive false-positive rate. The presented research incorporates both supervised and unsupervised machine learning algorithms, such as Random Forest (RF) and DBSCAN, into the Wazuh detection pipeline as a way to complement accuracy and operational efficiency. The framework can help to quickly detect unknown attack vectors and malicious behaviour based on ML-based classification by collecting real-time event data

of Wazuh agents, processing logs centrally via Elastic Stack and apply corresponding classification, ensuring threat response within a SOC with low latency [10]. Our Wazuh project shares several similarities with this approach. Like the study, we enhance Wazuh's detection capabilities beyond static rule sets by implementing custom YARA rules and anomaly detection models to identify malicious activity.

Both projects leverage real-time data collection and centralized processing to enable efficient alert generation and visualiza- tion. Additionally, our project incorporates external threat intelligence sources, such as VirusTotal, and custom dashboards for Linux and Windows agents, aligning with the ML-enhanced detection methodology to provide a more proactive, adaptive, and scalable security monitoring framework. An unsupervised model called ARCADE (Adversarially Regularized Convolutional Autoencoder) detects complex anomalies on network traffic. The model employs adversarial training to not only enhance the precision of the detection but also generate resistance to noisy information characteristics posing major advantages on implementing in real-time SOC systems [8]. The practice of using ML-based anomaly detection innovations has proven to be highly versatile in the detection of previously unknown attack patterns due to compatibility with advanced SIEM platforms.

As the threat landscape becomes more and more advanced, there were recently several studies on the integration of machine learning together with Wazuh and OpenSearch to enhance coverage and minimize false positives in SIEMs. In this case, [1] solves the problem of dealing with insider threats that can be very discreet and hard to monitor by applying the Random Cut Forest (RCF) algorithm in detecting high risk actions like egress of data, needing elevated privileges, and lateral issues. The method helped to lower false alerts by 35 percent and achieved 94-percent true positive rate, with most anomalies identified within three minutes on a production setting, by cross-referencing ML-generated anomaly scores with traditional rule-based alerts. Shifting our attention to the Wazuh project, it adds custom YARA rules and decoders, embedded anomaly detection, and even external threat intelligence, such as VirusTotal to the core SIEM functionality. Real-time dashboards also can give centralized visibility into Linux and Windows agents, and provide a more rapid and effective response. Our framework is a composite strategy that uses signature-based detection, machine learning, and intelligence- driven insights, thereby presenting a flexible, scalable and practical answer that can identify both current and future threats; closely bridging the gap in previous works through methods and advantages.

With its open-source SIEM platform, Wazuh can be used to perform log inspection and includes file integrity monitoring, intrusion detection and real-time alerting characteristics. Through its flexibility, Wazuh provides optimal performance with malware scanning as well as anomaly de- tention systems with options that include integration. Various research works confirm that Wazuh is effective in a variety of aspects of security. The studies reveal that Wazuh is an effective tool that allows consolidating threat data into valuable operational intelligence in apparel industry [6]. Wazuh demonstrated its features of real-time policy enforcement and threat warning in cloud security monitoring situations in a study analysis [2]. At the same time, a working solution showed the possibility of Wazuh to stop brute-force attacks in the form of active responses and Telegram alerts in enterprise systems [3]

Chapter 3

Methodology

The proposed system will be able to combine the YARA malware detection technology with the use of ML network anomaly detection through a common architectural framework. Aggregation of Alerts and their visualization via Wazuh SIEM dashboard involves the specification of the data flow pathways to find out how to transfer the alerts in the Detect modules. Figure:3.1 will present a wazuh distributed multinode configuration that will consist of at least two distinct agents (Linux and Windows) agents. All about Wazuh setup is discussed in section 3.1. The agents will be expected to generate logs during normal circumstances and in the circumstance of attack. We will initiate a YARA malware attack and will snag all packets with Wireshark/tcpdump. The packets are processed and sent out as .pcap file. We intend to use several files of format .pcap in this project.

Extracting these .pcap files is done, with the help of the scapy library, in Jupyter Notebook. We will then train unsupervised anomaly detection models with Scikit- learn. And additionally apply the Isolation Forest method along with One-Class SVM, which allows the goal of anomaly scoring. These scores will be transferred into JSON files and they will be included in the logs to be send to

The article Wazuh manager. Our project intends to have such a fully functional Integrated Threat Detection Framework Using YARA and Network Anomaly Detection with Wazuh Dashboard for SOC Operations as one of the biggest pieces of our project is the custom dashboard designing to represent these scores and the alert. Figure 3.1 shows summary figure of the full work on the project.

3.1 Wazuh

Wazuh is a SIEM and XDR platform that provides real-time security monitoring, identification of threats, and automated response and is offered as open-source software. It easily integrates with the Elastic Stack and is designed to support custom rules, decoders and dashboards to tailor to individual security operations. It has YARA-based malware detection, file integrity monitoring, vulnerability assessment, and the VirusTotal threat intelligence integration as its primary capabilities. Wazuh can also be used to perform intrusion detection, active response tools, compliance monitoring, and multi-node deployments, making the security solution comprehensive to the newer and more demanding SOC environment.

Wazuh can be installed in different ways based on the size of the environment and its needs. The all-in-one installation places the Wazuh manager, indexer, and dashboard on a single node. This setup works well for labs or small deployments. For production use, the distributed installation spreads components across different servers to improve performance and scalability. Wazuh also offers an installation assistant (wazuh-install.sh) that automates the setup with minimal user input. The step-by-step manual method allows

for full customization by installing and configuring each component separately. In larger businesses, a multi-node clustered setup is used where several indexers, managers, and dashboards run in high-availability mode. This flexibility ensures that Wazuh can meet the needs of both small and large SOC environments.

For this project, we are using the distributed multinode Wazuh Infrastructure. We are using Ubuntu 22.04 Jellyfish on Hyper-V to set up the Wazuh distributed infrastructure. We recognize VM1 as a Wazuh Indexer, and dashboard. VM2 as a Master Wazuh server and keeping the VM3 as the future Worker Wazuh server, We have also added three wazuh agents one in Linux and one in Windows.

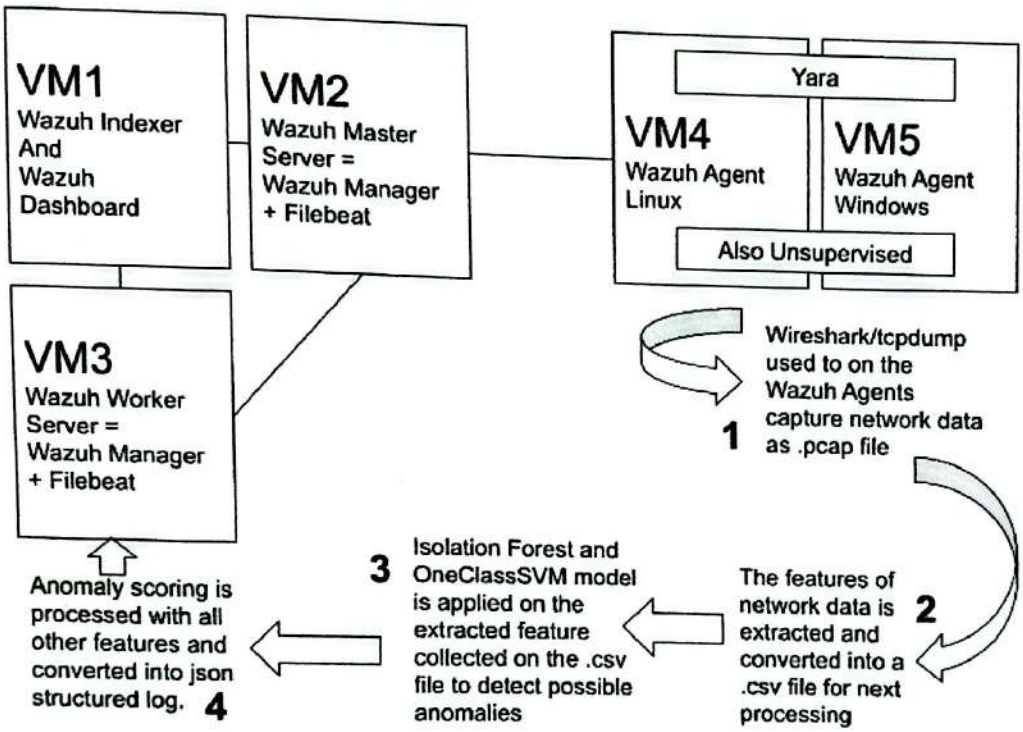

Figure 3.1: Our project at a glance

3.1.1 Wazuh Certification

Wazuh certificates are crucial for securing communication between components and ensuring data integrity and trust. They are created before installing components. This allows for the configuration of static IPs for fixed nodes and DNS names for dynamically assigned IPs. As a result, the deployment can work smoothly across any network. By using do- main names like wazuh-indexer.local, wazuh-manager.local, and wazuh-dashboard.local instead of direct IP addresses, the setup gains more flexibility and manageability in any network.

We downloaded the Wazuh installation assistant and the configuration file. We edited the config.yml. Finally, we ran the Wazuh installation assistant to generate the Wazuh cluster key, certificates, and passwords needed for installation.

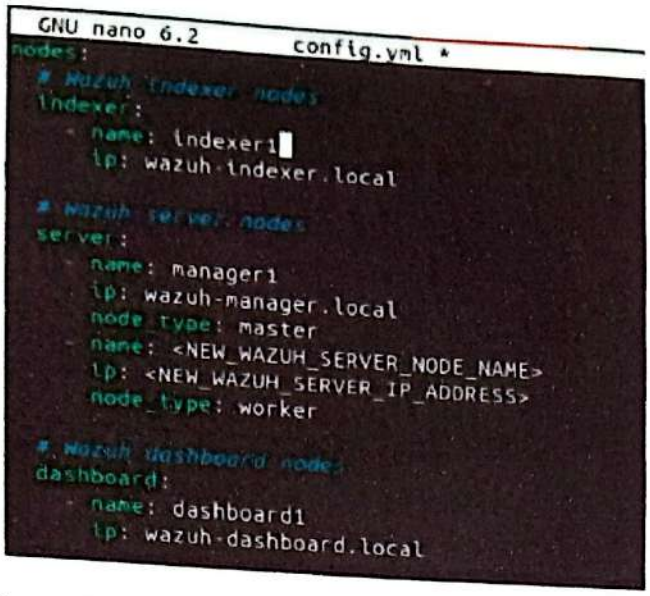

Figure 3.1.1: Configuring config.yml for our project.

```
curl -sO https://packages.wazuh.com/4.12/wazuh-install.sh
curl -sO https://packages.wazuh.com/4.12/config.yml
bash wazuh-install.sh --generate-config-files
```

All of the servers in the distributed deployment, including the Wazuh server, Wazuh indexer, and Wazuh dashboard nodes, should have the wazuh-install-files.tar file copied to them. The scp utility can be used to accomplish this.

3.1.2 Wazuh Indexer, Manager & Dashboard Installation

Wazuh Indexer

The Wazuh Indexer is the data storage and search engine part of the Wazuh platform, built on OpenSearch. It indexes, stores, and manages the security events, logs, and alerts from the Wazuh Manager. By allowing quick searches, queries, and correlations, the Wazuh Indexer helps analysts identify threats, investigate incidents, and gather insights. It acts as the foundation of the Wazuh architecture, providing the structured data layer that enables visualization and monitoring through the Wazuh Dashboard.

we download the Wazuh installation assistant. Next we run the Wazuh installation assistant and the node name to install and configure the Wazuh indexer. We have to make sure that the node name must be the same one used in config.yml for the initial configuration, for example, node-1 (indexer1).

```
curl -sO https://packages.wazuh.com/4.12/wazuh-install.sh
bash wazuh-install.sh --wazuh-indexer node-1
```

A Wazuh Cluster is a distributed setup where multiple Wazuh Manager nodes (one master and several workers) work together to improve scalability, performance, and high availability for large environments. Initializing cluster creation and testing the cluster installation.

Figure 3.1.2 (a): Wazuh indexer installation.

```
bash wazuh-install.sh --start-cluster
tar -axf wazuh-install-files.tar wazuh-install-files/wazuh-passwords.txt -O | grep -P "\'admin\'" -A 1
```

Running the following command to confirm that the installation is successful. Repla- ceing ¡ADMIN PASSWORD¿ with the password gotten from the output of the previous command. Replace ¡WAZUH INDEXER IP¿ with the configured Wazuh indexer IP address (iwazuh-indexer.local). And also checking if Wazuh cluster has been successfully installed.

```
curl -k -u admin:<ADMIN PASSWORD>https://<WAZUH INDEXER IP>:9200
curl -k -u admin:<ADMIN PASSWORD>https://<WAZUH INDEXER IP>:9200/_cat/nodes?v
```

Figure 3.1.2 (b): Wazuh cluster installation.

Wazuh Server

The Wazuh server analyzes the data received from the agents, triggering alerts when it detects threats and anomalies. This central component includes the Wazuh manager and Filebeat. We have 2 Wazuh server in our project one is Master and another is Worker. We have configured our certification such a way that in future if needed we can add more Wazuh server nodes easily.

To install Wazuh server first we need to make sure that a copy of the wazuh-install-files.tar, created during the initial configuration step, is placed in the working directory. where we will download the Wazuh installation assistant. Then we need to run the Wazuh installation assistant followed by the node name to install the Wazuh server. The node name must be the same one used in config.yml for the initial configuration, for example, wazuh-1 (in our case it will be manager1).

```
curl -sO https://packages.wazuh.com/4.12/wazuh-install.sh
bash wazuh-install.sh --wazuh-server wazuh-1
```

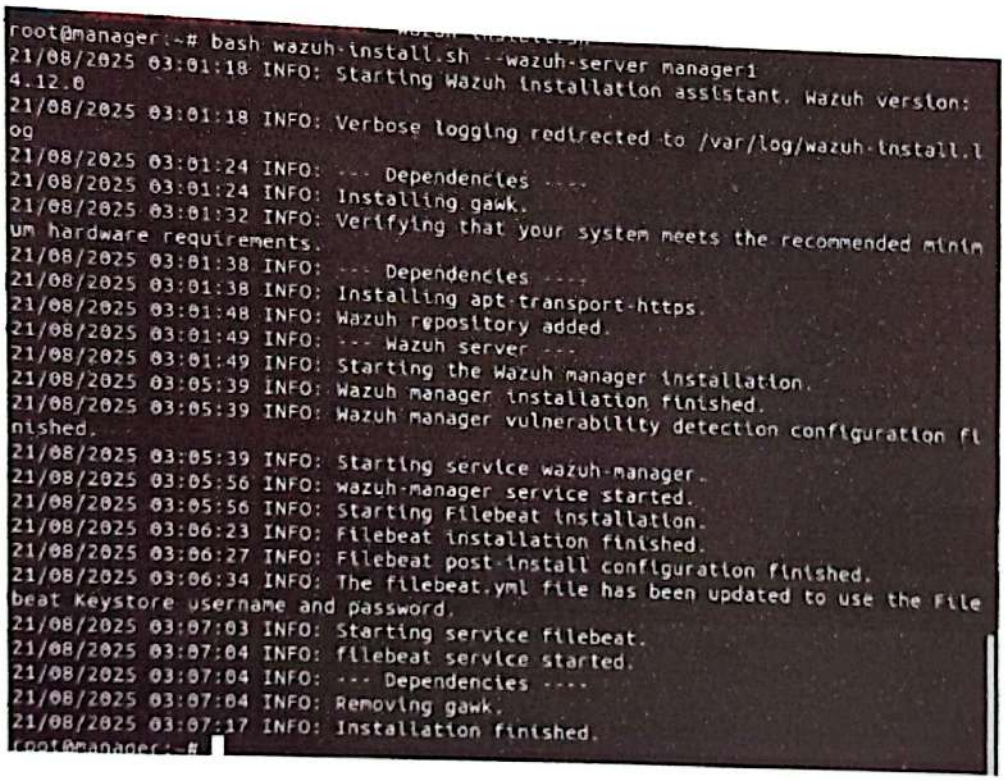

Figure 3.1.2 (c): Wazuh server installation.

Wazuh Dashboard

The Wazuh Dashboard is a web-based interface built on OpenSearch Dashboards that provides real-time visibility into security events, logs, vulnerabilities, and compliance data. It offers customizable visualizations and reports, enabling analysts to monitor threats, investigate incidents, and respond effectively. Wazuh also supports the creation of custom rules and decoders, allowing organizations to tailor detection to their specific environments, applications, and threat models. This flexibility ensures more accurate alerts and reduces false positives.

Key features of the dashboard include active response management, integration with threat intelligence (e.g., VirusTotal), vulnerability detection, file integrity monitoring, regulatory compliance checks, and centralized multi-agent monitoring. Together, these capabilities make the Wazuh Dashboard a powerful tool for both operational security and compliance management.

Since our Wazuh indexer and dashboard are in the same VM so we have skiped the process of downloading the Wazuh installation assistant. We also ignored copying of the wazuh-install-files.tar file, created during the initial configuration step, as we already have it in our working directory. So we straight move to the next step of running the Wazuh installation assistant with the node name to install and configure the Wazuh dashboard. The node name must be the same one used in config.yml for the initial configuration, for example, dashboard (in our case its dashboard1). Once the Wazuh installation is completed, the output shows the access credentials and a message that confirms that the installation was successful.

```
bash wazuh-install.sh --wazuh-dashboard dashboard
INFO: --- Summary ---
    INFO: You can access the web interface
    https :// <WAZUH_DASHBOARD_IP_ADDRESS>
    User : admin
    Password : <ADMIN_PASSWORD>
INFO: Installation finished.
```

Our Wazuh all components are now installed and configured. But still have one step left, we need to have all passwords that the Wazuh installation assistant generated in the wazuh-passwords.txt file inside the wazuh-install-files.tar archive. We need to print them by running the following command:

```
tar -O -xvf wazuh-install-files.tar wazuh-install-files/
wazuh-passwords.txt
```

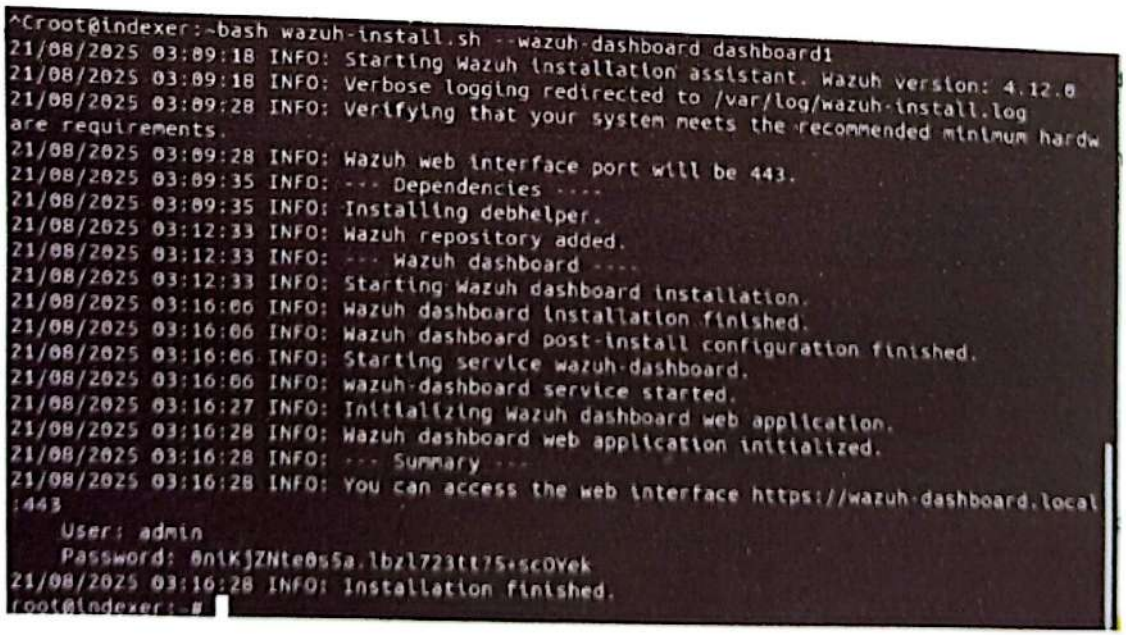

Figure 3.1.2 (d): Wazuh dashboard installation.

Now we access the Wazuh web interface with our admin user credentials. This is the default administrator account for the Wazuh indexer and it allows us to access the Wazuh dashboard.

URL: https :// <WAZUH_DASHBOARD_IP_ADDRESS>
Username : admin
Password : <ADMIN_PASSWORD>

Figure 3.1.2 (e): Wazuh dashboard login page.

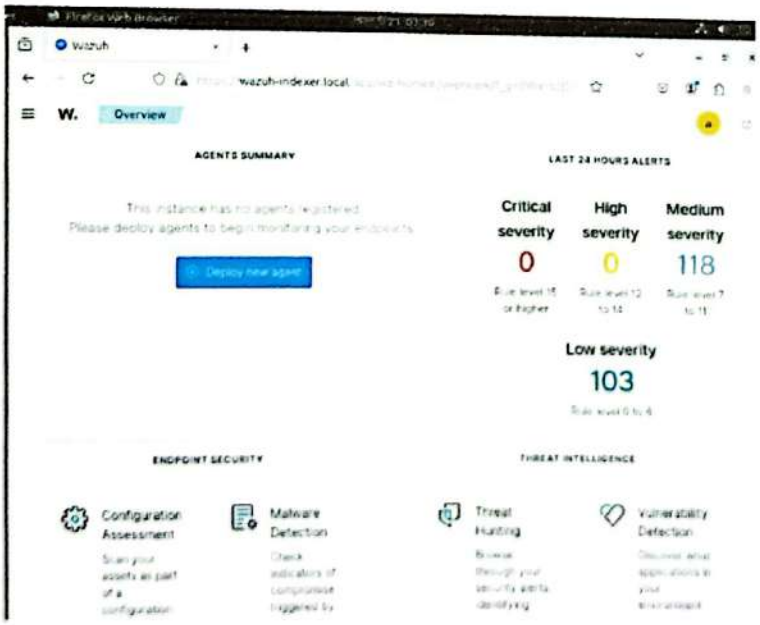

Figure 3.1.2 (f): Wazuh dashboard overview.

3.1.3 Wazuh Agent Creation

The Wazuh agent is multi-platform and runs on the endpoints that the user wants to monitor. It communicates with the Wazuh server, sending data in near real-time through an encrypted and authenticated channel. The agent was developed considering the need to monitor a wide variety of different endpoints without impacting their performance. It is supported on the most popular operating systems.

Figure 3.1.3 (a): Types of OS that allows to have Wazuh agent

Linux Agent

The agent runs on the host you want to monitor and communicates with the Wazuh server, sending data in near real-time through an encrypted and authenticated channel. Firstly, we add the Wazuh repository to download the official packages. Then we install the GPG key, followed by adding the repository and updating the packager information.

```
curl -s https://packages.wazuh.com/key/GPG-KEY-WAZUH |
gpg --no-default-keyring --keyring gnupg-ring:/usr/share
/keyrings/wazuh.gpg --import && chmod 644 /usr/share/
keyrings/wazuh.gpg

echo "deb [signed-by=/usr/share/keyrings/wazuh.gpg] https://
packages.wazuh.com/4.x/apt/ stable main" | tee -a /etc/apt/
sources.list.d/wazuh.list

apt-get update
```

Deploy a Wazuh agent To deploy the Wazuh agent on the endpoint, we need to select package manager and edit the WAZUH_MANAGER variable to contain your Wazuh manager IP address or hostname. Here we will add our wazuh-manager.local instead of IP address. And finally we enable and start the Wazuh agent service.

```
WAZUHMANAGER="wazuh-manager.local" apt-get install wazuh-agent
systemctl daemon-reload
systemctl enable wazuh-agent
systemctlstart wazuh-agent
```

Figure 3. together with thei (b): Linux agent installation.

Windows Agent

To start the installation process, download the Windows installer from Wazuh's web- site. To deploy the Wazuh agent on your endpoint, choose one of the command shell alternatives and edit the WAZUH_MANAGER variable so that it contains the Wazuh manager IP address or hostname. We can either user the CLI or the GUI method to install Windows agent. Both method is easy and flexible. Moreover, To deploy the Wazuh agent on the Windows endpoint, we can either CMD or Powershell and edit the WAZUH MANAGER variable so that it contains the Wazuh manager IP address or hostname. Using PowerShell:

```
. \ wazuh–agent –4 .12 .0 –1 . msi /q WAZUHMANAGER= " 10 . 0 . 0 . 2 "
NET START Wazuh
```

The installation process is now complete, and the Wazuh agent is successfully installed and configured. And we start the Wazuh agent by running the NET START Wazuh command. Once started, the Wazuh agent will start the enrollment process and register with the manager.

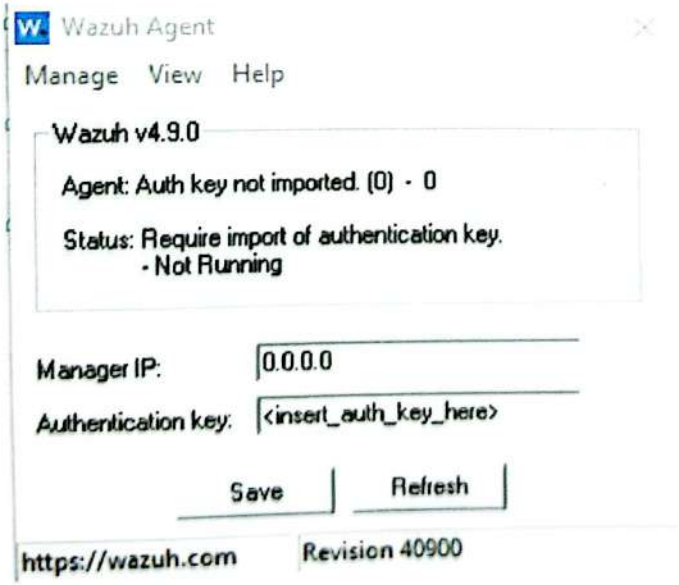

Figure 3.1.3 (c): Windows agent installation (GUI version).

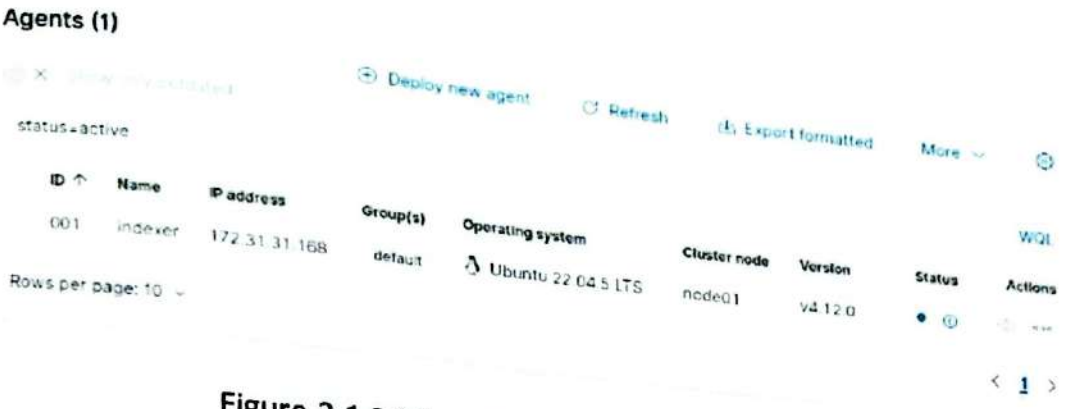

Figure 3.1.3 (d): Linux agent on Wazuh dashboard.

3.2 YARA

YARA (Yet Another Ridiculous Acronym) is generic malware detection and classification tool commonly used in the malware industry. The software can be used to achieve some forms of human readable rules defining patterns in files, processes, or memory, which can be highly effective in threat hunting and digital Forensics. The primary characteristics of YARA are: rule-based detection based on strings and regular expressions, the capability to scan files and running processes, support of logical operators, and support of extensions in form of modules, including PE or ELF analysis. Its advantages consist in the fact that it is lightweight, fast, can be easily customized, and that it is widely accepted by the security community, yet there are limitations, including vulnerability to the quality of the rules, frequent updates as a necessity, and lower likeliness of being effective against highly obfuscated malware results.

When integrated with Wazuh, YARA significantly enhances detection capabilities by scanning monitored files and directories against custom or community YARA rules. De- tected matches are sent to the Wazuh Manager as alerts, indexed in the Wazuh Indexer, and visualized in the Dashboard for further analysis. This integration benefits security teams by enabling real-time malware detection, correlation of malicious activity with other security events, and improved incident response. By combining YARA's flexible rule-based scanning with Wazuh's SIEM features, organizations gain a more comprehen- sive approach to threat detection and response.

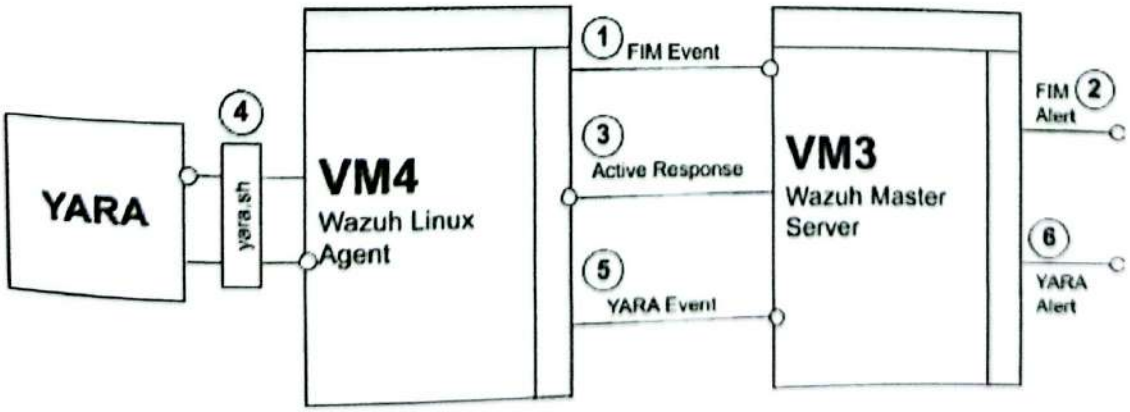

Figure 3.2: How YARA integration works on Wazuh.

3.2.1 YARA Installation

We will configuration YARA in our Linux endpoint. And to perform the following steps to install YARA, and we also need to configure the Active Response and FIM modules in Wazuh Linux endpoint as well.

Step 1: Download and install YARA.

```
sudo apt update
sudo apt install -y make gcc autoconf libtool libssl-dev
pkg-config jq
sudo curl -LO https://github.com/VirusTotal/yara/archive/
v4.2.3.tar.gz
sudo tar -xvzf v4.2.3.tar.gz -C /usr/local/bin/ && rm -f
v4.2.3.tar.gz
cd /usr/local/bin/yara-4.2.3/
sudo ./bootstrap.sh && sudo ./configure && sudo make && sudo
make install && sudo make check
```

Step 2: We type yara to find out if YARA is running properly. And if we get the following output that means YARA has been successfully installed.

```
yara
Output
yara: wrong number of arguments
Usage: yara [OPTION]... [NAMESPACE:]RULES_FILE... FILE | DIR
| PID
Try '--help' for more options
```

Step 3: Next we download the YARA detection rules from its distinct sources.

```
sudo mkdir -p /tmp/yara/rules
sudo curl 'https://valhalla.nextron-systems.com/api/v1/get' \
-H 'Accept: text/html,application/xhtml+xml,application/xml;
q=0.9,*/*;q=0.8' \
-H 'Accept-Language: en-US,en;q=0.5' \
--compressed \
-H 'Referer: https://valhalla.nextron-systems.com/' \
-H 'Content-Type: application/x-www-form-urlencoded' \
-H 'DNT: 1' -H 'Connection: keep-alive' -H 'Upgrade-Insecure
-Requests: 1' \
--data 'demo=demo&apikey=1111111111111111111111111111111111
1111111111111111111111111111&format=text' \
-o /tmp/yara/rules/yara_rules.yar
```

Step 4: We need to create a yara.sh script in the /var/ossec/active-response/bin/ directory. This is necessary for the Wazuh-YARA Active Response scans. (full code is added in Appendix A)

```
read INPUT_JSON
YARA_PATH=$(echo $INPUT_JSON | jq -r .parameters.extra_args[1])
YARA_RULES=$(echo $INPUT_JSON | jq -r .parameters.extra_args[3])
FILENAME=$(echo $INPUT_JSON | jq -r .parameters.alert.syscheck.path)
# Set LOG FILE path
LOG_FILE="logs/active-responses.log"
#------------ Analyze parameters ------------#
if [[ ! $YARA_PATH ]] || [[ ! $YARA_RULES ]]
then
    echo "wazuh-yara: ERROR - Yara active response error.
    Yara path and rules parameters are mandatory." >> ${LOG_FILE}
    exit 1
fi
#------------ Main workflow ------------#
# Execute Yara scan on the specified filename
yara_output="$("${YARA_PATH}"/yara -w -r "$YARA_RULES" "$FILENAME")"
if [[ $yara_output != "" ]]
then
    # Iterate every detected rule and append it to the LOG FILE
    while read -r line; do
        echo "wazuh-yara: INFO - Scan result: $line" >> ${LOG_FILE}
    done <<< "$yara_output"
fi
exit 0;
```

Step 5: Changing yara.sh file owner to root:wazuh and file permissions to 0750. After that we have to add the directory location within the ¡syscheck¿ block of the Wazuh agent /var/ossec/etc/ossec.conf configuration file to monitor the /tmp/yara/malware directory. And then we restart Wazuh agent since we edited the configuration file.

```
<directories realtime="yes">/tmp/yara/malware</directories>
sudo systemctl restart wazuh-agent
```

Step 6: Next, We have to add custom rules to the /var/ossec/etc/rules/local_rules.xml file in the manager's VM. We can also do this from the dashboard. The custom rules detect FIM events in the monitored directory. They also alert when the YARA integration finds malware.

```
<group name="syscheck,">
    <rule id="100550" level="8">
        <if_sid>550</if_sid>
        <field name="file">/tmp/yara/malware/</field>
        <description>File modified in /tmp/yara/malware/
        directory.</description>
    </rule>
```

```
    <rule id="100551" level="8">
        <if_sid>554</if_sid>
        <field name="file">/tmp/yara/malware/</field>
        <description>File added to /tmp/yara/malware/
        directory.</description>
    </rule>
</group>

<group name="yara,">
    <rule id="109000" level="0">
        <decoded_as>yara_decoder</decoded_as>
        <description>Yara grouping rule</description>
    </rule>
    <rule id="109001" level="12">
        <if_sid>109000</if_sid>
        <match>wazuh-yara: INFO - Scan result:</match>
        <description>File "$(yara_scanned_file)" is a
        positive match. Yara rule: $(yara_rule)
        </description>
    </rule>
</group>
```

Step 7: We follow almost the same process to add the following decoders to the Wazuh server /var/ossec/etc/decoders/local_decoder.xml file in the Wazuh manager's VM. The custom decoder allows for extracting the information from YARA scan results.

```
<decoder name="yara_decoder">
    <prematch>wazuh-yara:</prematch>
</decoder>

<decoder name="yara_decoder1">
    <parent>yara_decoder</parent>
    <regex>wazuh-yara: (\S+) - Scan result: (\S+) (\S+)
    </regex>
    <order>log_type, yara_rule, yara_scanned_file</order>
</decoder>
```

Step 8: In the last step of fully functionable YARA malware detection system we add the following configuration to the Wazuh server /var/ossec/etc/ossec.conf configuration file. Then we restart the Wazuh manager. This configures the Active Response module to trigger after the rule 100550 and 100551 are fired.

```
<ossec_config>
    <command>
        <name>yara _linux </name>
        <executable >yara . sh </executable >
        <extra _args >-yara_path  /usr/local/bin -yara rules
        /tmp/yara/rules/yara rules . yar </extra _args >
        <timeout _allowed >no</timeout _allowed >
    </command>

    <active -response >
        <disabled >no</disabled >
        <command>yara_linux </command>
        <location >local </location >
        <rules_id >100300,100301 </rules_id >
    </active -response >
</ossec_config>

sudo systemctl  restart wazuh-manager
```

Now we have a fully functional YARA malware detection system in our Linux agent. We have demonstrated an attack and the output of the alert generated is discussed in the section no. 4.3.

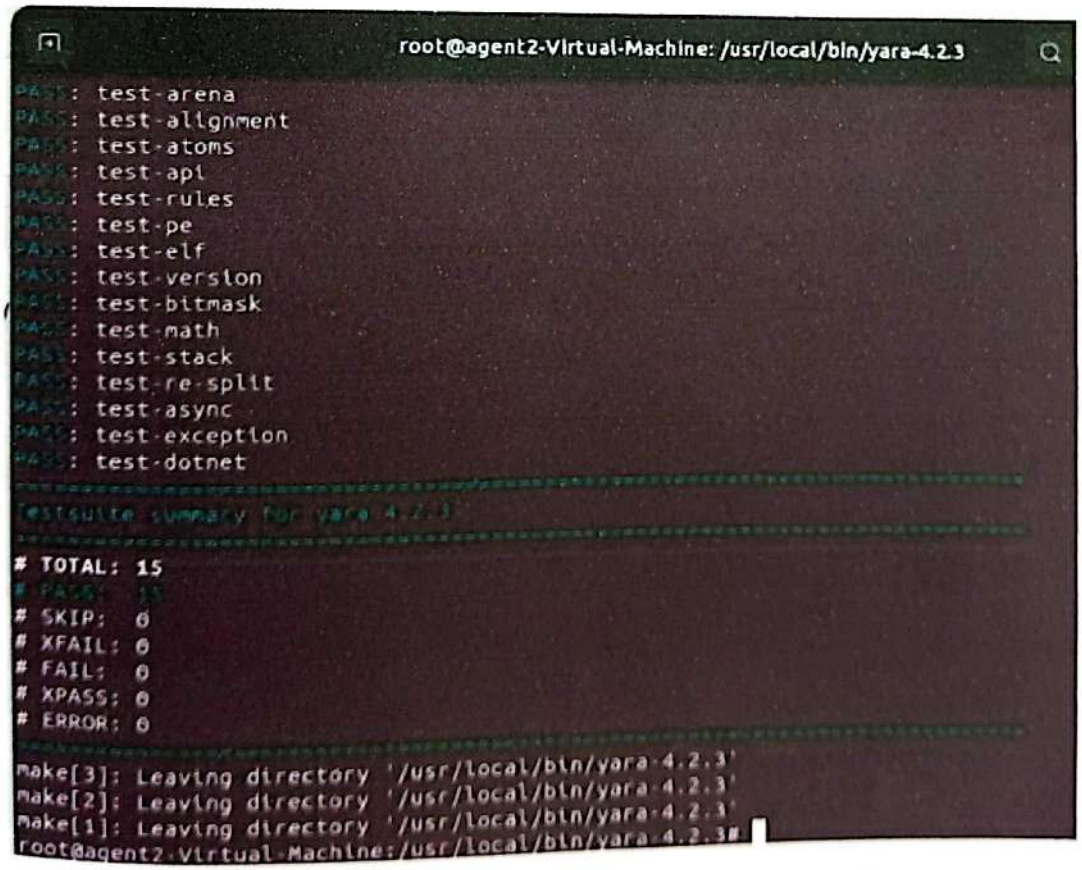

Figure 3.2.1: Successfully installed YARA in linux agent.

3.3 Anomaly Detection

In this project, the integration of Isolation Forest and One-Class SVM plays a critical role in strengthening the anomaly detection capability of our Wazuh-based SOC frame- work. While Wazuh, with its rules, decoders, and YARA integration, is excellent at identifying known threats, modern attacks often involve unknown or subtle deviations in network traffic and system behavior that static rules may miss. To address this gap, we incorporated machine learning models—Isolation Forest and One-Class SVM—designed specifically for unsupervised anomaly detection. These models do not require labeled datasets, which makes them highly suitable for real-world environments where malicious activity often lacks prior examples.

Isolation Forest works by randomly partitioning data points and isolating anomalies, which are typically easier to separate than normal points. This makes it efficient for detecting outliers in large, high-dimensional datasets such as live network traffic logs. One-Class SVM, on the other hand, creates a boundary around normal behavior and flags anything outside this boundary as abnormal. Together, these models complement each other: Isolation Forest provides speed and scalability, while One-Class SVM offers precision in defining the "normal profile" of system behavior. By combining them, we increase detection accuracy and minimize false positives, making our project stand out compared to traditional Wazuh-only deployments.

The role of these models is crucial in maintaining the chain of analysis that leads from raw network events on Wazuh Agents (VM4 and VM5) to meaningful alerts visualized in the Wazuh Dashboard. The models process live traffic data on the Linux agent, transform it into structured anomaly detection insights, and output results as JSON logs, which are then fed back into Wazuh for correlation and visualization. Without this machine learning layer, the system would lack adaptive intelligence against zero-day threats and unusual attack patterns, making this integration central to the project's success.

To demonstrate this, we implemented and tested the models in a controlled environ- ment using Jupyter Notebook version 7.4.5 on the Linux agent (VM4). This allowed us to experiment interactively, validate results in real time, and ensure that anomalies were correctly transformed into logs consumable by Wazuh. Overall, the combination of Isolation Forest and One-Class SVM not only strengthens our threat detection pipeline but also makes the project unique by bridging classical SIEM capabilities with advanced machine learning-based anomaly detection.

3.3.1 Python Environment Setup

In our work, we have found Jupyter Notebook very useful to create a flexible interactive environment in Linux agent. It enabled us to easily incorporate Python3 code to implement One-Class SVM, Isolation Forest, feature extraction and generate JSON logs as well as install necessary libraries. It was only possible because the notebook allowed combing code and their outputs and visualizations, which, in turn, made it possible to plot and analyse anomaly detection results in real-time. Such flexibility allowed to perform testing, debugging, and validation efficiently to achieve an efficient workflow where the machine learning outputs were convertible into JSON logs that were uploaded to Wazuh to be analyzed and monitored by it

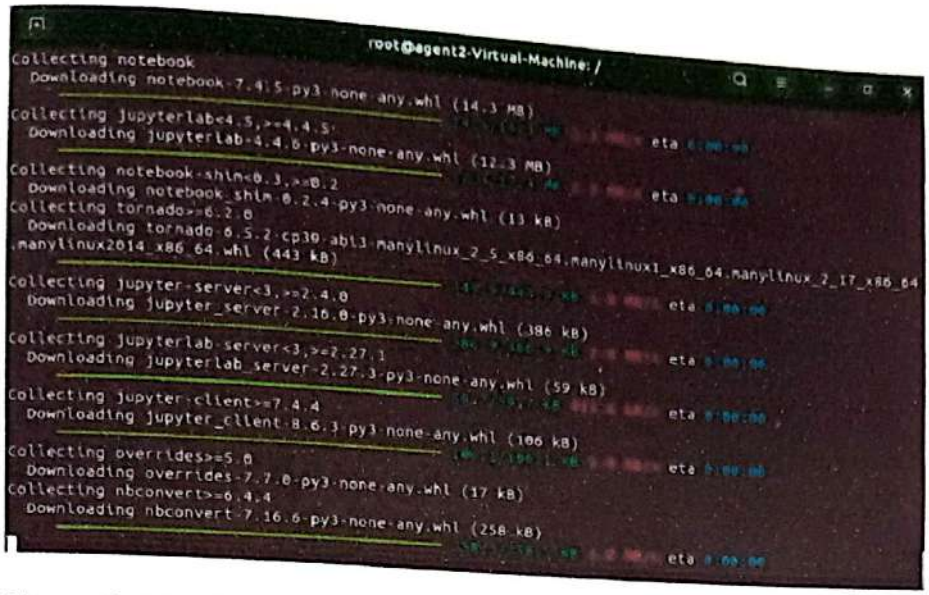

Figure 3.3.1 (a): Jupyter notebook installation process in Linux agent.

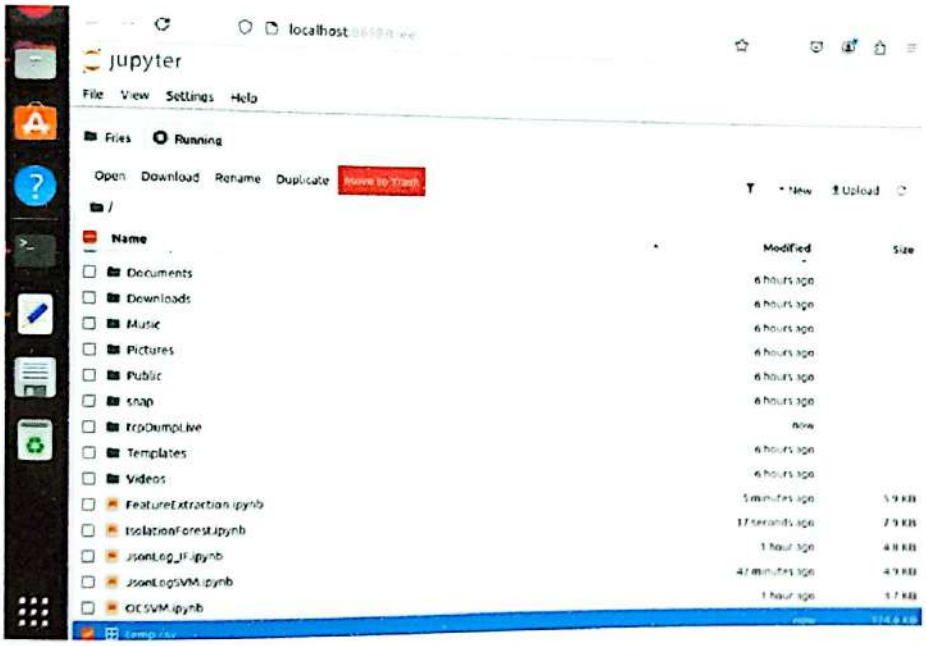

Figure 3.3.1 (b): Successfully installed jupyter notebook.

Pandas

Pandas is an effective open source Python package that is used to manipulate and analyze the data. It offers advanced data structures including DataFrames and Series, which help users to conveniently work with structured-data like tables, CSV files, JSON logs, and time-series data. Pandas were useful in our project when processing extracted data in network packets and establishing Log DataFrames and export outcomes to JSON logs compatible with Wazuh integration. It also enabled us to preprocess data to be used by Isolation Forest and One-Class SVM models and to efficiently manage rolling datasets with regard to anomaly detection.

Scapy

Scapy is a rich Python packet modification module. It enables one to capture, create and decode network packets and transmit them, which is highly beneficial during network analysis, pen testing, and research. In the same line of research, Scapy can be applied to read .pcap files, extract features of packets and analyze the traffic flow to detect anomalies in the same way we did ourselves in projects like ours.

Matplotlib

Matplotlib is a popular Python plotting library with the aim of making visualizations of data. It allows to create static, type-challenged, and animated graphs like line graphs, bar charts and scatter graphs. In our project, the Matplotlib is employed to display the results of Isolation Forest and One-Class SVM models so that anomalies can be more easily examined and reviewed.

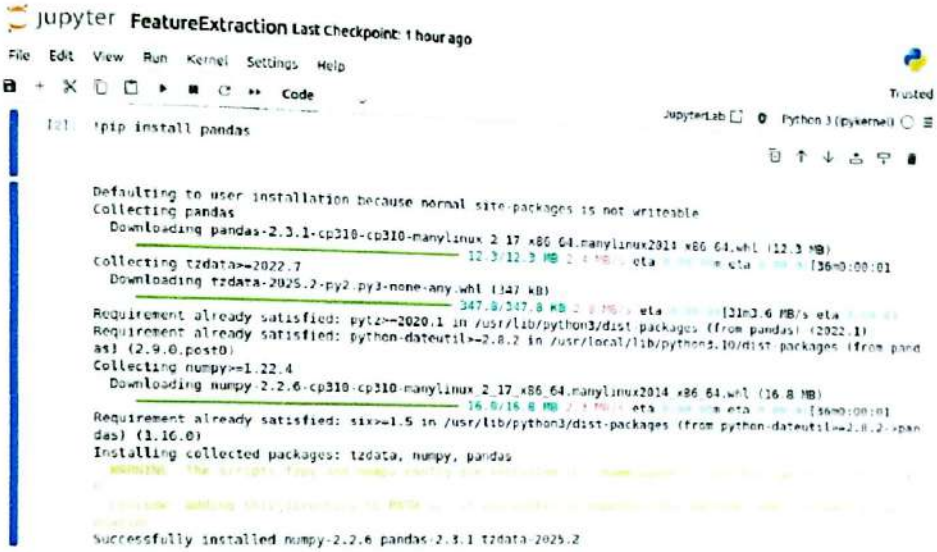

Figure 3.3.1 (c): Installing pandas.

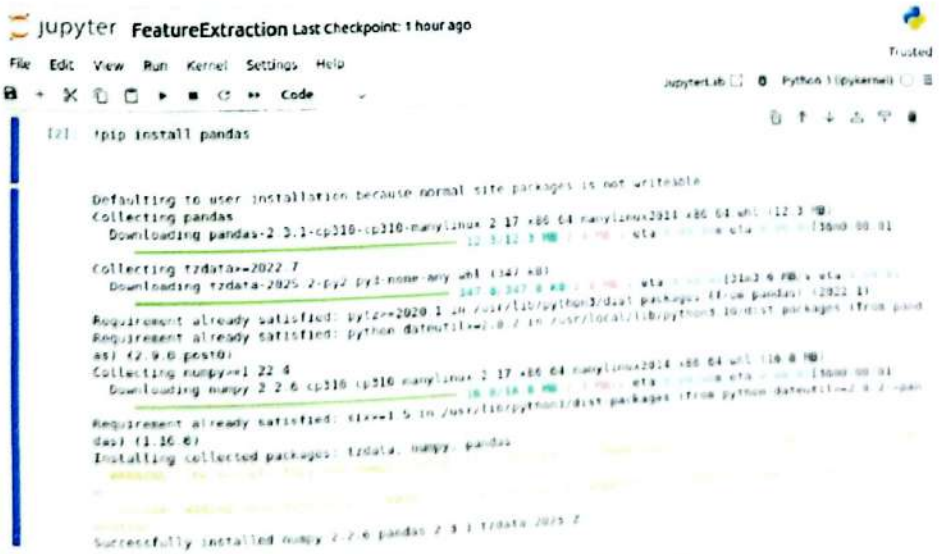

Figure 3.3.1 (d): Installing Scapy.

3.3.2 Wireshark & tcpdump

tcpdump and Wireshark are the basic instruments to capture the raw network traffic on the agents in our project. We interacted with the Linux agent through command-line based tcpdump to capture packets efficiently and with the windows agent by using a graphical traffic analyzer Wireshark. These tools are optioned to guarantee that all packets passing through the end nodes are captured and recorded, thus serving as the basis of our anomalous detection line of action.

The captured data is stored in a capture.pcap file, which we configured with a max- imum size of 50 MB. Once this limit is reached, the file overwrites old entries, ensuring continuous logging without exhausting disk space. This rolling mechanism is crucial for maintaining up-to-date network data while keeping storage usage under control. The .pcap files collected through tcpdump and Wireshark are then processed for feature ex-traction, feeding structured data into our machine learning models (Isolation Forest and One-Class SVM).

tcpdump and Wireshark cannot be overemphasised in this project, because they contain the input raw layer of the anomaly detection. Without an efficient and reliable packet capture, the machine learning models would not be enjoying substantial data to learn and the entire chain that results in the generation of JSON logs in Wazuh would report a failure.

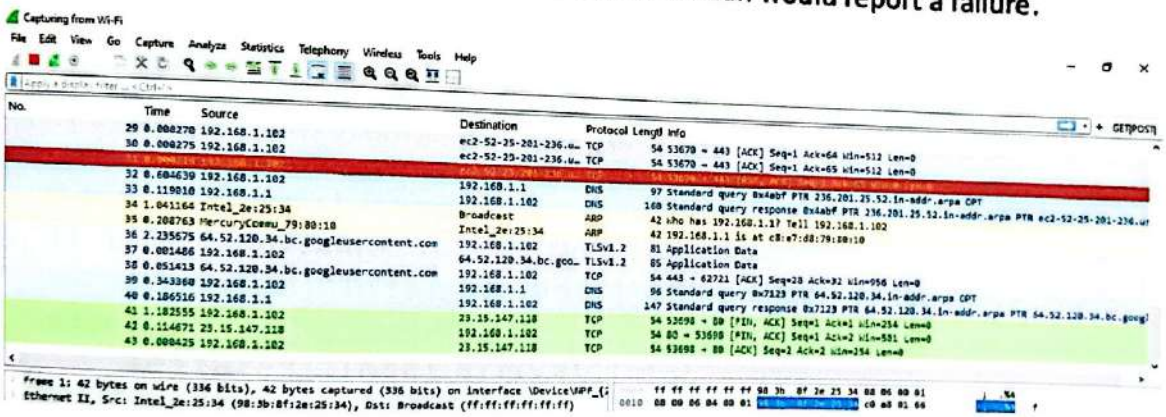

Figure 3.3.2 (a): Running Wireshark in Windows agent.

Figure 3.3.2 (b): Running tcpdump in Linux agent.

3.3.3 Feature Extraction

In our project, feature extraction is the bridge between raw network packet captures and the anomaly detection models. After collecting traffic data in .pcap files using tcpdump and Wireshark, we processed these files to extract meaningful attributes such as source and destination IP addresses, ports, protocol type, packet size, and flow duration. These features were then structured into a tabular format suitable for machine learning analysis.

The extraction process transforms unstructured network traffic into quantifiable values that the Isolation Forest and One-Class SVM models can analyze effectively. Without this step, the models would not be able to differentiate between normal and abnormal patterns, as raw packet data is too granular and noisy. By selecting relevant features, we reduced complexity, minimized redundancy, and improved the efficiency of our anomaly detection pipeline.

The following function takes a single packet and converts it into a structured dictionary of features. It records the timestamp, inter-arrival time (time since the last packet), source/destination IPs, protocol type, packet length, and TCP/UDP ports and flags. Non-IP packets are skipped. This prepares raw network packets for analysis or aggregation.

```
def extract_features(pkt):
    global last_timestamp
    features = {}

    if IP not in pkt:
        return None  # skip non-IP packets

    ip = pkt[IP]
    features["timestamp"] = pkt.time

    # inter-arrival time
    if last_timestamp is not None:
        features["inter_arrival"] = pkt.time - last_timestamp
    else:
        features["inter_arrival"] = 0
    last_timestamp = pkt.time

    features["src_ip"] = ip.src
    features["dst_ip"] = ip.dst
    features["protocol"] = ip.proto
    features["length"] = len(pkt)

    #TCP/UDP ports and flags
    if ip.proto == 6 and TCP in pkt:
        tcp = pkt[TCP]
        features["srcport"] = tcp.sport
        features["dst_port"] = tcp.dport
        features["flags"] = str(tcp.flags)
    elif ip.proto == 17 and UDP in pkt:
        udp = pkt[UDP]
        features["srcport"] = udp.sport
        features["dstport"] = udp.dport
        features["flags"] = "NONE"
```

```
    else:
        features["src_port"] = None
        features["dst_port"] = None
        features["flags"] = "NONE"
    return features
```

This part calculates aggregated metrics over the last WINDOW_SIZE packets stored in packet buffer. It computes the packet rate, byte rate, and the number of unique source and destination ports. These aggregated features help to summarize traffic behavior over a small time window, which is useful for detecting anomalies or monitoring network load.

```
def compute_window_features():
    """Compute aggregated features over sliding window."""
    if not packet_buffer:
        return {}

    df = pd.DataFrame(packet_buffer)
    window_features = {
        "pkt_rate": len(df) / (df["inter_arrival"]
        .sum() + 1e-6),  # packets per sec
        "byte_rate": df["length"].sum() / (df["inter_arrival"]
        .sum() + 1e-6),  # bytes per sec
        "unique_src_ports": df["src_port"].nunique(),
        "unique_dst_ports": df["dst_port"].nunique(),
    }
    return window_features
```

This function writes the latest packet features to the CSV file, always adding the newest row at the top. It ensures the CSV does not exceed the maximum size (50MB) by removing the oldest rows if necessary. Using a temporary file avoids corruption during updates and safely replaces the original CSV once it's trimmed and updated.

```
def write_to_csv_top(features):
    """Prepend the latest features to CSV, keep max size 50MB."""
    new_row = pd.DataFrame([features])

    if os.path.exists(CSV_FILE):
        # Read existing CSV
        df_existing = pd.read_csv(CSV_FILE)
        # Prepend new row
        df_combined = pd.concat([new_row, df_existing],
        ignore_index=True)
    else:
        df_combined = new_row
..... more code .....
    os.replace(temp_file, CSV_FILE)
```

Full code is added in the end of this project in Appendix A.

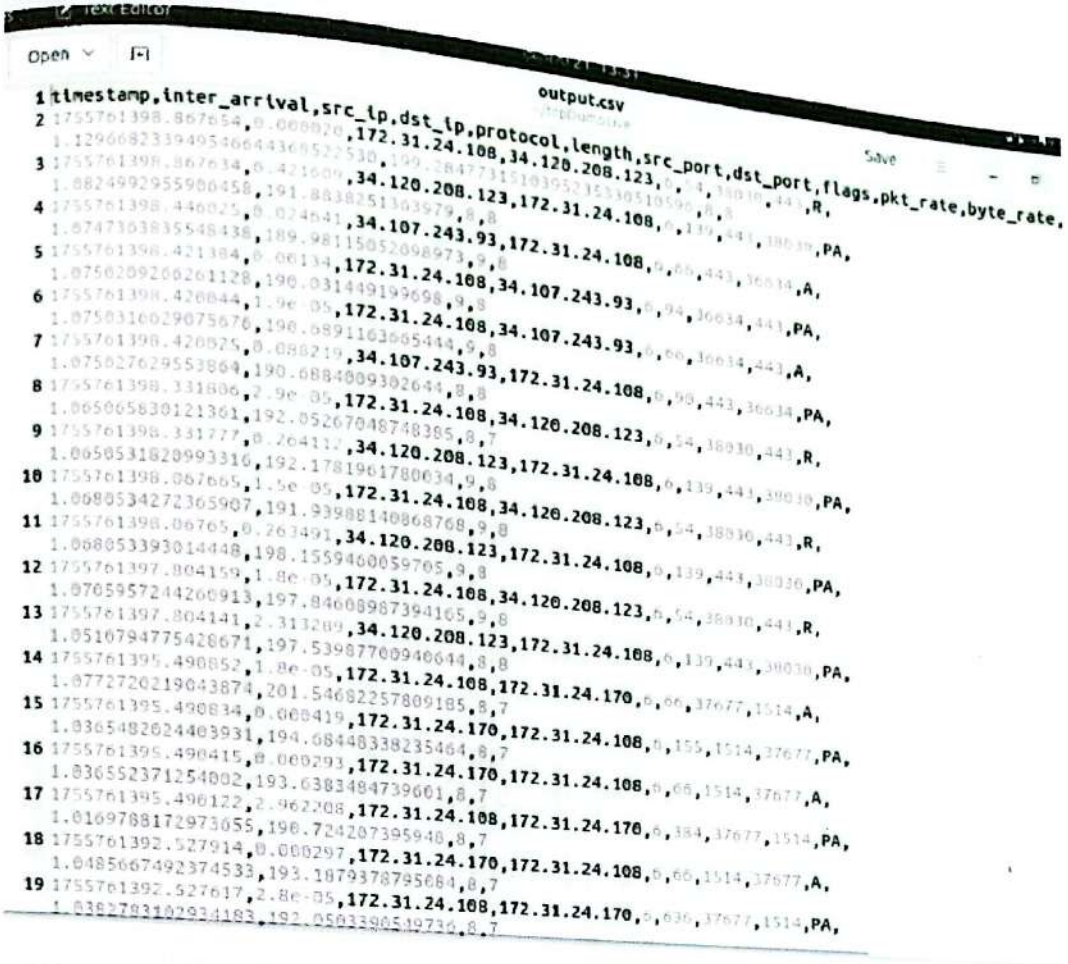

Figure 3.3.3: Successfully extracted features form capture.pcap and saved into output.csv

3.3.4 Isolation Forest Live Plotting

There is an unsupervised learning algorithm, called Isolation Forest, which is considered to be effective in anomaly detection. As opposed to conventional techniques that involved the modeling of normal behavior, Isolation Forest isolates instances that do not perform as expected by isolating these points in multiple random decision trees. Anomalies are less in number and clear in nature, which makes the process efficient and highly scalable, provided the profiles of the high-dimensional data sets and even real-time streams are to be detected.

In our SOC project, Isolation Forest plays a critical role in enhancing threat detection capabilities. It allows the system to automatically analyze live network traffic and system logs, detecting abnormal patterns, unusual behaviors, or potential intrusions that may not match predefined signatures. By complementing YARA-based malware detection, it provides a layered and proactive defense mechanism, generating real-time alerts and actionable insights on the Wazuh dashboard. This not only reduces the reliance on manual monitoring but also strengthens overall security operations, enabling faster and more informed responses to emerging threats.

The following code is the main loop continuously checks for the existence of the OUTPUT CSV file. It reads only the first row from the CSV (which is the latest packet feature row from the live capture) and selects only numeric columns for anomaly detection. If the row contains no numeric data, it skips to the next iteration. This ensures the Isolation Forest only processes relevant features.

```
while True:
    if os.path.exists(OUTPUT_CSV):
        try:
            # Read only the first row of output.csv
            row = pd.read_csv(OUTPUT_CSV, nrows=1)
            X_row = row.select_dtypes(include=['float64',
            'int64']).dropna()
            if X_row.empty:
                continue
```

An IsolationForest model is initialized with a contamination rate of 5% (i.e., expected proportion of anomalies). It is then fit on the single row of numeric features and predicts whether the row is an anomaly (-1) or normal (1). This adds a new column anomaly to mark the row as anomalous or not. Normally, Isolation Forest is trained on multiple rows, but here it's applied to a rolling live stream for demonstration purposes.

```
# Fit Isolation Forest on this single row
model = IsolationForest(contamination=0.05, random_state=42)
model.fit(X_row)
row['anomaly'] = model.predict(X_row)
```

The newly detected row is appended to the rolling anomaly queue. This deque is then converted to a DataFrame to save all recent anomalies to ANOMALY CSV for persistence. Using a rolling queue ensures the CSV doesn't grow indefinitely while keeping recent history for visualization and further analysis. These data is plotted in the Jupyter notebook in section 4.1. Full code is added in Appendix A.

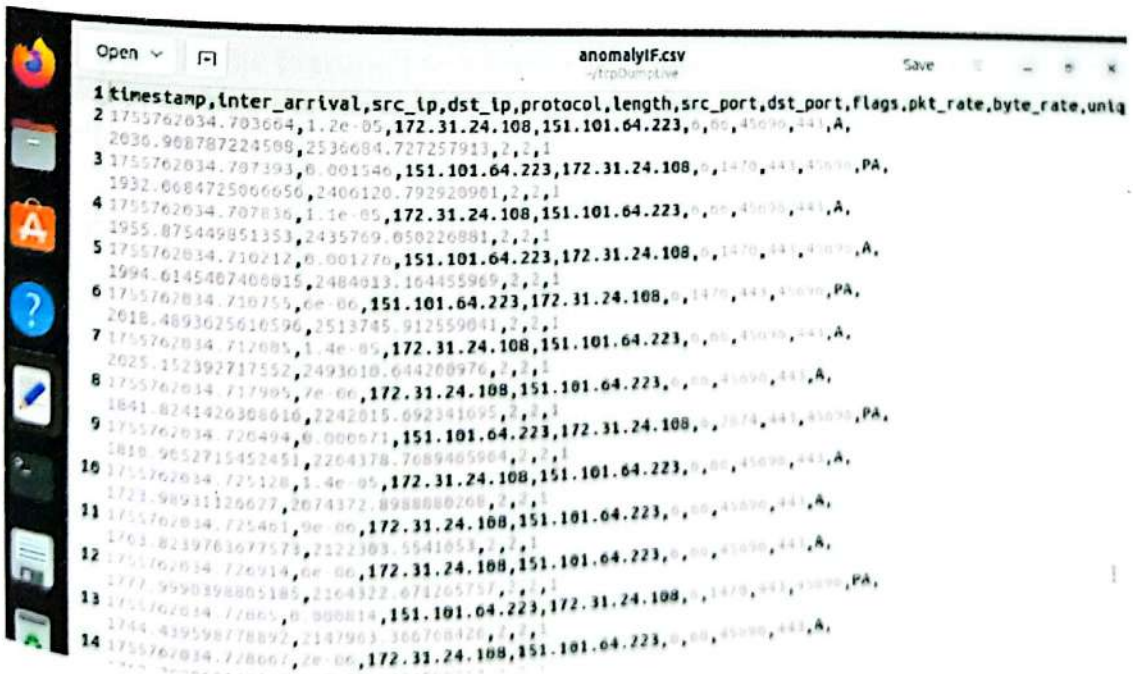

Figure 3.3.4: Isolation forest model data.

3.3.5 One-Class SVM Live Plotting

One-Class Support Vector Machine (One-Class SVM) is an unsupervised machine learning algorithm which has gained widespread use in detecting anomalies. In comparison with traditional SVM, which classifies d different classes, One-Class SVM targets at learning a boundary of normal data in a high dimension space. It determines a decision function that forms the area in which most normal data should exist and any other point falling beyond that boundary would be seen as an abnormality. This property qualifies it especially well in applications involving cybersecurity and network intrusion detection where the rare event of interest cannot be easily labeled anomalous.

In our SOC/Wazuh project, One-Class SVM was integrated as part of the anomaly detection module to monitor live network data. The algorithm was trained on extracted features from packet captures, modeling the baseline of normal traffic patterns. During live monitoring, incoming network data was continuously compared against this learned boundary, and any deviation was flagged as a potential anomaly. Visualization of the results was carried out in Jupyter Notebook, enabling us to clearly track the distinction between normal and anomalous behavior. The complete implementation code for One-Class SVM has been included in Appendix B for reproducibility.

The inclusion of One-Class SVM is highly significant to the success of our project, as it enhances the reliability of anomaly detection and improves real-time monitoring capabilities within Wazuh. Although the majority of readings were classified as normal, which is expected in unsupervised monitoring, the model's ability to highlight subtle deviations adds substantial value to our framework. By combining One-Class SVM with Isolation Forest and YARA-based malware detection, our system provides a multi-layered defense mechanism that strengthens proactive threat identification and ensures a more resilient SOC operation. Similar to the Isolation Forest code this is the main loop continuously

checks if the OUTPUT_CSV file exists. It reads only the first row of the CSV (the latest network traffic features) and selects only numeric columns. If no numeric values are available, it skips the iteration. This step ensures that the anomaly model processes meaningful data only.

```
while True:
    if os.path.exists(OUTPUT_CSV):
        try:
            # Read only the first row of output.csv
            row = pd.read_csv(OUTPUT_CSV, nrows=1)
            X_row = row.select_dtypes(include=['float64',
            'int64']).dropna()
            if X_row.empty:
                continue
```

Before applying One-Class SVM, the numeric data is standardized using StandardScaler, since SVMs are sensitive to feature scales. Then, a One-Class SVM model with RBF kernel is trained on the single scaled row. The parameter nu=0.05 specifies the expected anomaly proportion. Finally, a prediction (1 = normal, -1 = anomaly) is added as a new anomaly column to the row.

```
# Scale the data (important for One-Class SVM)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_row)

# Fit One-Class SVM on this single row
model = OneClassSVM(kernel='rbf', gamma='auto', nu=0.05)
        model.fit(X_scaled)
row['anomaly'] = model.predict(X_scaled)
```

The processed row is appended to the rolling deque anomaly_queue. This deque is then converted into a DataFrame and saved to ANOMALY_CSV. Storing only the rolling window (last 500 anomalies) prevents the anomaly log file from becoming too large while still keeping recent history for analysis. These data is plotted in the Jupyter notebook in section 4.1. Full code is added in Appendix A.

```
# Append to rolling anomaly queue
anomaly_queue.append(row.iloc[0])

# Convert deque to DataFrame for saving and visualization
df_anomaly = pd.DataFrame(anomaly_queue)
df_anomaly.to_csv(ANOMALY_CSV, index=False)
```

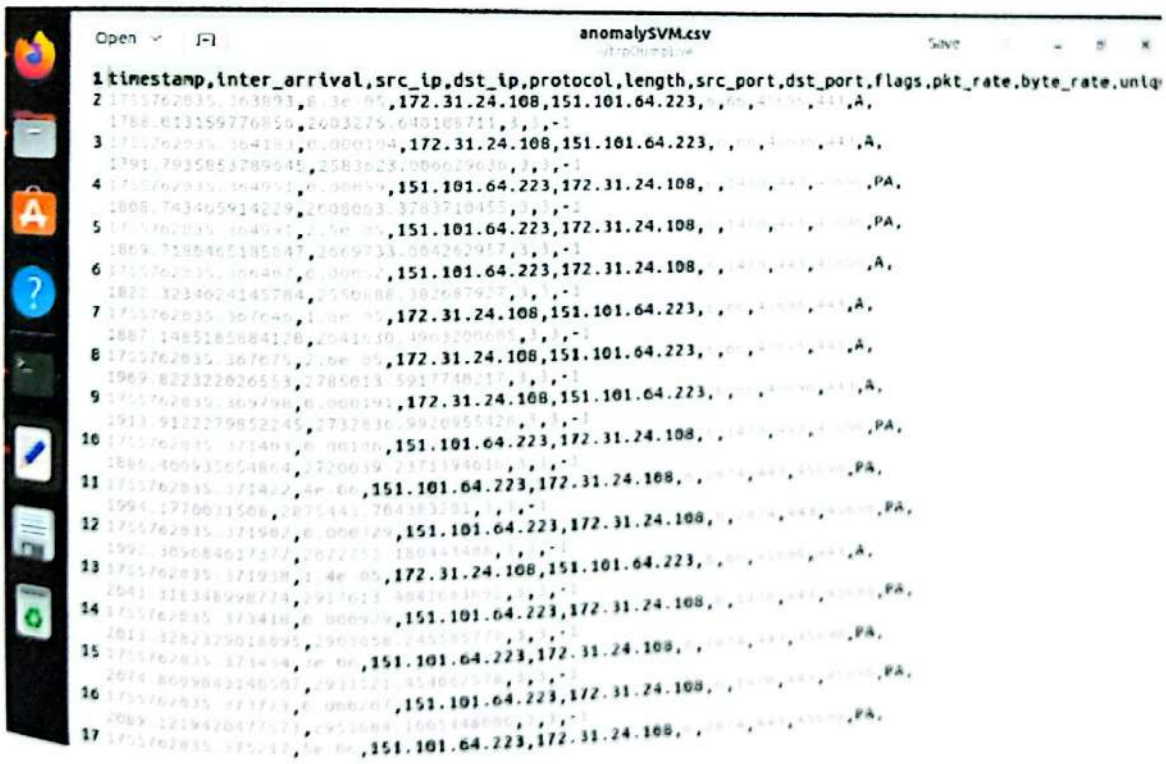

Figure 3.3.5: One-Class SVM model data.

3.3.6 Live JSON Log Creation

In our project, both Isolation Forest and One-Class SVM were employed to generate anomaly scores from live network traffic data. These algorithms, though different in their approaches, served the common purpose of identifying data points that deviated from normal network behavior. Isolation Forest calculated anomaly scores based on the average path length required to isolate data points within its random tree structures, with shorter paths indicating higher likelihoods of anomalies. One-Class SVM, on the other hand, generated anomaly scores by measuring the distance of data points from the learned boundary of normal traffic patterns in high-dimensional space. Together, these methods provided complementary perspectives on abnormal behavior, enabling us to not only classify data as normal or anomalous but also to assign quantitative anomaly scores that reflected the severity of the deviation.

To ensure that the outputs from these algorithms could be seamlessly integrated into our security monitoring framework, the anomaly scores were combined with the extracted network traffic features and converted into JSON-structured logs. Each JSON entry included critical metadata, such as timestamp, source and destination IP addresses, ports, protocol type, packet size, and the corresponding anomaly scores from both Isolation Forest and One-Class SVM. This structured representation of data ensured consistency, interoperability, and machine-readability, making it highly suitable for ingestion by SIEM platforms.

The conversion of results into JSON format was a crucial step for our project. JSON logs not only standardized the way anomaly data was stored and transmitted but also made it possible to integrate seamlessly with the Wazuh server. Once generated, these JSON logs were forwarded to Wazuh server using Wazuh agent (fig3.24), ensuring that the anomaly detection outputs from our models were available alongside traditional log sources within the SIEM environment. By doing so, we enabled centralized monitor- ing, correlation, and visualization of machine learning–based anomaly detection results directly on the Wazuh dashboard.

Network Data → Feature Extraction → ML Models (IF + OCSVM) → JSON Log → Wazuh Agent → Wazuh Server → Wazuh Dashboard

This integration was of significant importance to our project. By feeding anomaly scores and enriched network features into Wazuh, we enhanced its native monitoring and alerting capabilities with advanced, AI-driven detection. This allowed us to move beyond signature-based detection methods and incorporate behavior-based intelligence, ensuring that both known and previously unseen threats could be identified and acted upon in real time. The approach ultimately contributed to building a robust, multi-layered SOC framework, combining machine learning with traditional security analytics to improve threat visibility, reduce false positives, and support proactive security operations.

The following code is this function builds a standardized JSON event from the result got in Isolation Forest and One-Class SVM model data. It extracts IPs, protocol, anomaly score, and prediction values (with fallback options). It also adds a UTC timestamp and the detector name. The resulting dictionary (event) is what gets logged to Wazuh. Full code can be found in Appedix A.

```
def build_event(row_dict):
# Required fields (with fallbacks)
src_ip   = _first_non_null(row_dict, ["src_ip", "source_ip"])
dst_ip   = _first_non_null(row_dict, ["dst_ip", "destination_ip"])
protocol =_first_non_null(row_dict, ["protocol", "proto"])

# prediction: prefer explicit 'prediction'; else 'anomaly';
else default 1
prediction =_int_from(row_dict, ["prediction", "anomaly",
"if_pred"], default=1)

# anomaly_score: prefer explicit; else derive -1 for anomalies,
1 for normal
score =_float_from(row_dict, ["anomaly_score", "score",
"if_score"],
                   default=(-1.0 if prediction == -1 else 1.0))
event = {
    "timestamp": datetime.utcnow().strftime("%Y-%m-%dT%H:%M:%S"),
    "src_ip": src_ip,
    "dst_ip": dst_ip,
    "protocol": protocol,
    "detector": DETECTOR_NAME,
    "anomaly_score":score,
    "prediction": prediction
}
return event
```

And then we have the flush_log() function writes the current buffer of anomaly events to LOG_FILE in JSON lines format (one event per line). This ensures the log is always updated with the latest anomalies, replacing older entries if the buffer exceeds its rolling size.

```
def flush_log():
with open(LOG_FILE, "w") as f:
    for e in log_buffer:
        f.write(json.dumps(e) + "\n")
```

The Wazuh agent is responsible for collecting, processing, and forwarding log data to the Wazuh manager. In our project, once the anomaly detection module produced the JSON-structured logs containing extracted network features and anomaly scores, these logs were written into a designated local file on the endpoint where the Wazuh agent was installed. The Wazuh agent continuously monitors such log files through its log collection module, which supports multiple formats including plain text, JSON, and syslog.

When the JSON log file is updated with new entries, the Wazuh agent reads the new content and forwards it to the Wazuh manager over an encrypted channel (using

TCP/UDP with TLS by default). On the manager side, the incoming logs are parsed using Wazuh's decoders and rules. Since our logs were structured in JSON format, the fields (such as timestamp, source IP, destination IP, protocol, and anomaly scores) could be extracted directly and mapped to meaningful attributes. These parsed events were then indexed and stored in the Wazuh indexer, making them available for visualization and correlation within the Wazuh dashboard.

In Wazuh, the <localfile>directive in the agent configuration specifies which files the agent should monitor and forward to the manager. In our project, the JSON logs gen- erated by Isolation Forest and One-Class SVM contained network features and anomaly scores. By setting <localfile>to the path of the JSON file and <log format>to JSON, the agent continuously monitored the file for new entries and forwarded them securely to the Wazuh manager (fig 3.6 (a)). There, the logs were parsed by decoders and rules, allow- ing the anomaly data to be indexed, correlated, and visualized on the Wazuh dashboard. The <localfile>thus acted as the key link between our machine learning outputs and the centralized SOC monitoring environment.

```
<localfile>
  <log_format>json</log_format>
  <location>/home/agent2/tcpDumpLive/anomaly_if.log</location>
</localfile>

<localfile>
  <log_format>json</log_format>
  <location>/home/agent2/tcpDumpLive/anomaly_svm.log</location>
</localfile>
```

Figure 3.3.6 (a): Wazuh agent sending JSON log to Wazuh server.

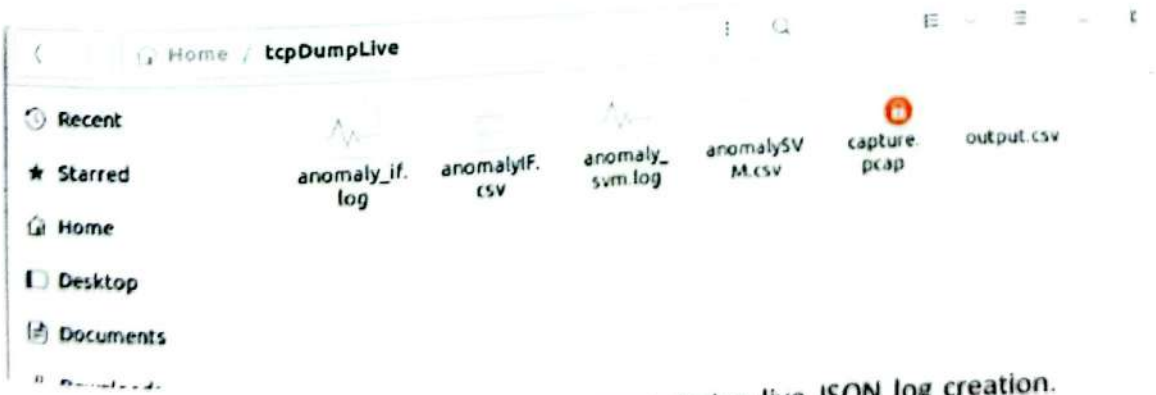

Figure 3.3.6 (b): List of files created during live JSON log creation.

Chapter 4

Results and Testing

4.1 Unsupervised Monitoring and Alert Generation

4.1.1 Isolation Forest Result Analysis

In the following fig 4.1.1 (a) results obtained from unsupervised monitoring of network data using Isolation Forest indicate that the majority of readings were classified as normal, which is consistent with expectations in such environments where anomalies are relatively rare. The visualization of the detection process and outcomes was carried out in Jupyter Notebook, providing clear graphical insights into the model's performance. For reference and reproducibility, the complete implementation code has been included in Appendix A. We used custom rules to generate alerts in the Wazuh dashboard (fig 4.3). The results that we collected is explained in this section.

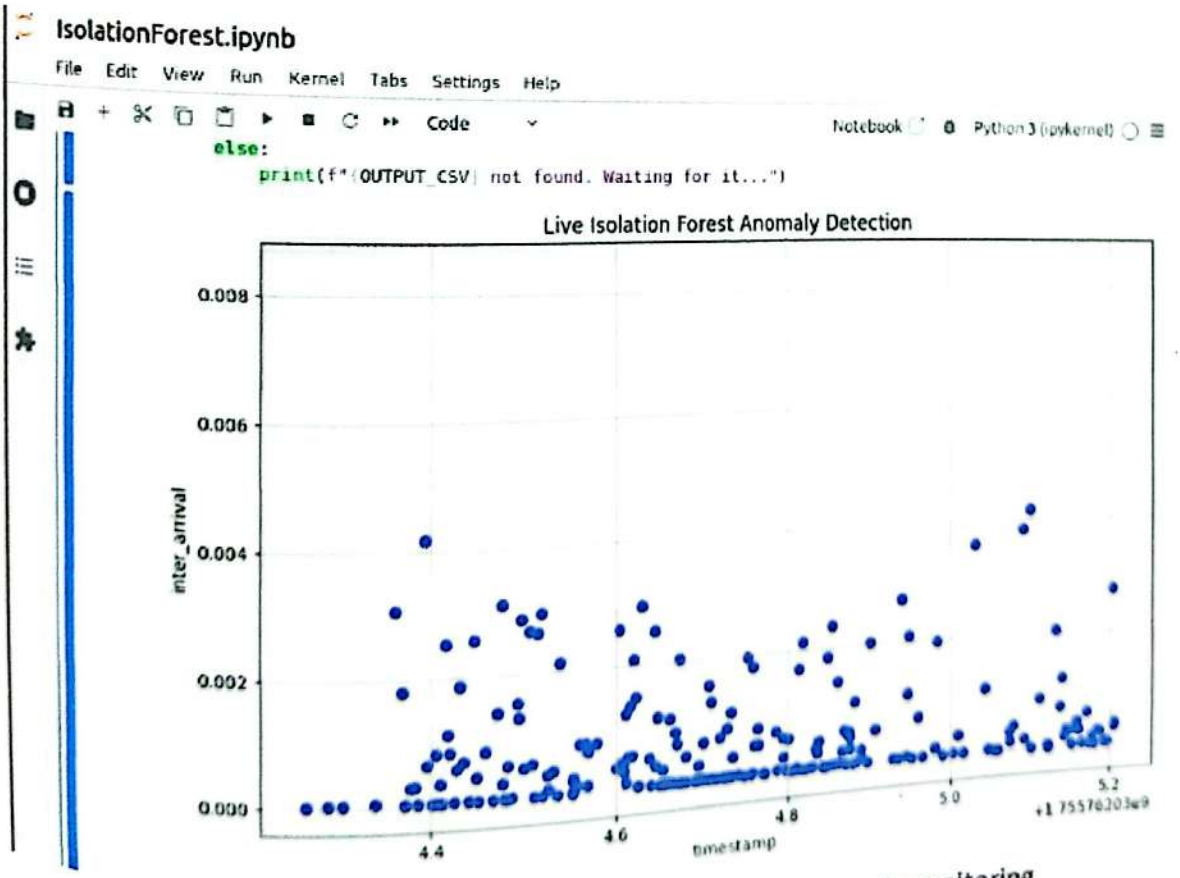

Figure 4.1.1 (a): Isolation Forest in unsupervised monitoring.

The alert was generated by Isolation Forest because of the way the algorithm interprets traffic patterns captured live from tcpdump. Isolation Forest is designed to identify anomalies by isolating points that deviate from the general distribution of data. In this case, even though the anomaly score is 1.0 and the prediction is 1 (indicating the event is considered normal by the model), the Wazuh rule set is configured to generate alerts whenever the detector reports activity, regardless of whether the event was classified as anomalous or not. This ensures that all evaluated traffic is logged and flagged for visibility.

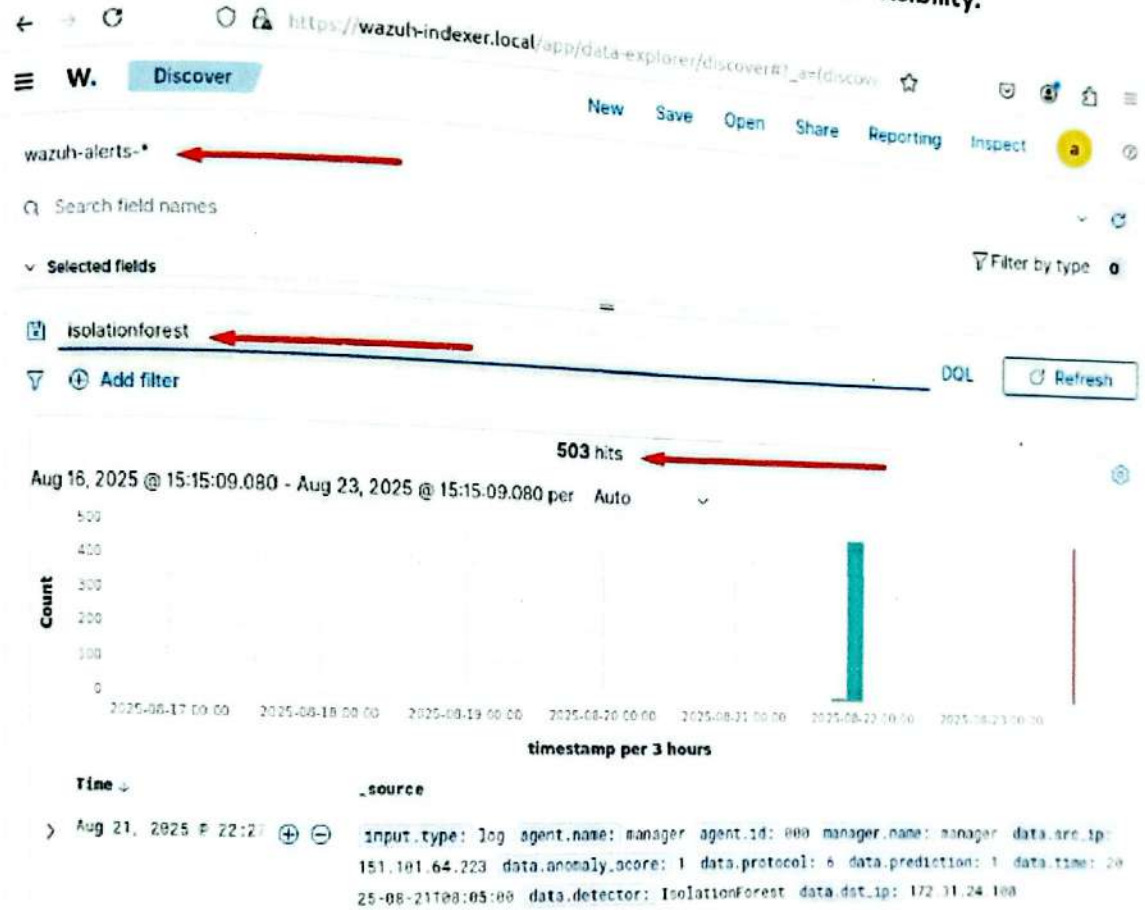

Figure 4.1.1 (b): Isolation Forest generated alerts in unsupervised monitoring.

Another factor is that Isolation Forest works by building random partitions of the data. Because the live data was collected continuously, every point is treated as part of the anomaly detection pipeline. This means that both anomalous (-1) and normal (1) classifications are still reported into Wazuh. As a result, even "normal" data points — shown in blue on the visualization — are logged with an associated score and rule match, which explains why all points appear as alerts.

Finally, the ruleset configuration plays a key role. In this case, the rule with ID 100555 is designed to fire whenever Isolation Forest produces an event, without filtering strictly on the prediction = -1 condition. This explains why alerts were generated for all captured packets, not just the red-labeled anomalies. Essentially, Wazuh is treating the entire stream of Isolation Forest output as valuable security telemetry and creating alerts to ensure visibility across all traffic samples.

local_rules.xml

```
<group name="anomaly_detection">
  <!-- Rule to detect anomaly prediction with anomaly score >= 1.0 -->
  <rule id="100555" level="10">
    <match>IsolationForest</match> <!-- Match the prediction value of 1 -->
    <description>Potential Anomalous Activity Detected (Isolation Forest)</description>
  </rule>
</group>

<group name="anomaly_detection">
  <!-- Rule to detect anomaly prediction with anomaly score >= 1.0 -->
  <rule id="100556" level="10">
    <match>OneClassSVM</match> <!-- Match the prediction value of 1 -->
    <description>Potential Anomalous Activity Detected (OneClassSVM)</description>
  </rule>
</group>
```

Figure 4.1.1 (c): Wazuh custom rules for both Isolation Forest and One-Class SVM.

4.1.2 One-Class SVM Result Analysis

The results obtained from unsupervised monitoring of network data using One-Class SVM show that the majority of readings were classified as normal, which aligns with expecta- tions since anomalies are relatively rare in the monitored environment. The visualization of the detection process and outcomes was performed in Jupyter Notebook, providing clear graphical insights into the model's performance. For reference and reproducibility, the complete implementation code has been included in Appendix A.

The behavior of the One-Class SVM (Support Vector Machine) model in this case highlights the differences between how it and Isolation Forest detect anomalies. In the first event, the One-Class SVM assigned an anomaly score of -1.0 and a prediction of -1, which both indicate that the event was classified as normal. In the context of One-Class SVM, negative values typically mean that the data point falls within the learned boundary of normal behavior. Since the event aligned closely with the expected data profile, no alert was triggered.

This outcome is consistent with how One-Class SVM operates. The algorithm is trained to learn the shape of "normal" data, and only those points that deviate signifi- cantly from this boundary are considered anomalous. In this case, the event in question did not cross the threshold required for anomaly detection. The model essentially viewed it as fitting well within the established pattern of traffic or system behavior.

In contrast, Isolation Forest approaches anomaly detection differently. By isolating data points using random partitions in decision trees, it is often more sensitive to devi- ations, even if they are subtle. As a result, Isolation Forest may flag unusual behavior that One-Class SVM considers normal. This explains why the same event was flagged as an anomaly by Isolation Forest but not by One-Class SVM.

The lack of anomaly detection by One-Class SVM may be attributed to several factors. If the model was trained on very homogeneous normal data, its boundary may have become too broad, allowing borderline cases to be classified as normal. Additionally, the sensitivity of One-Class SVM is strongly influenced by hyperparameter such as the kernel and the nu parameter, which controls the fraction of data expected to be outliers.

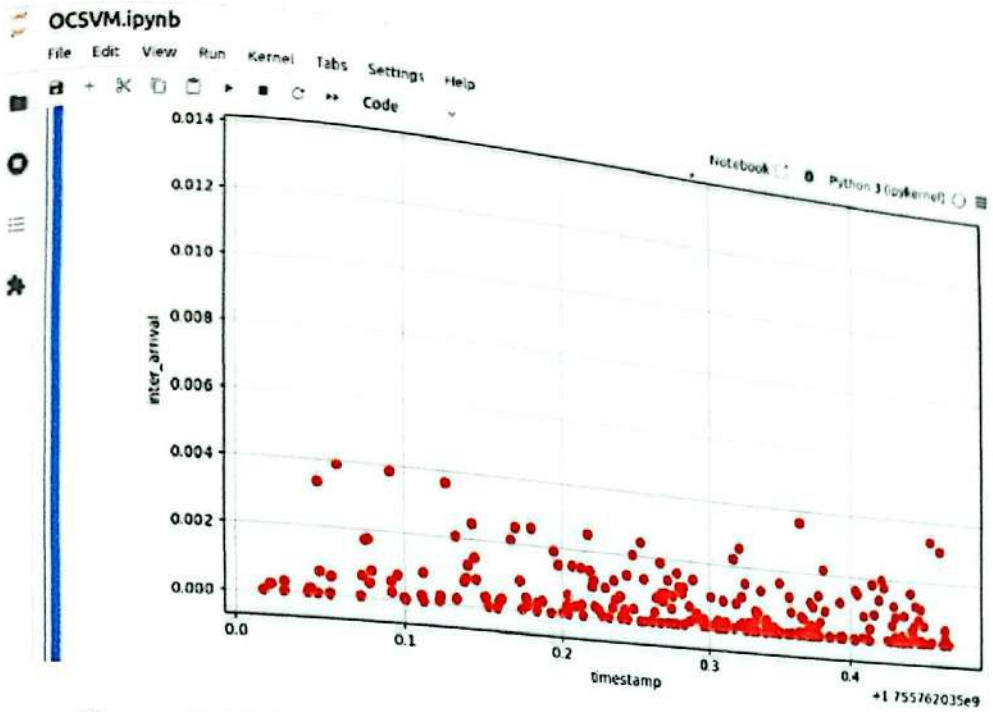

Figure 4.1.2 (a): One-Class SVM in unsupervised monitoring.

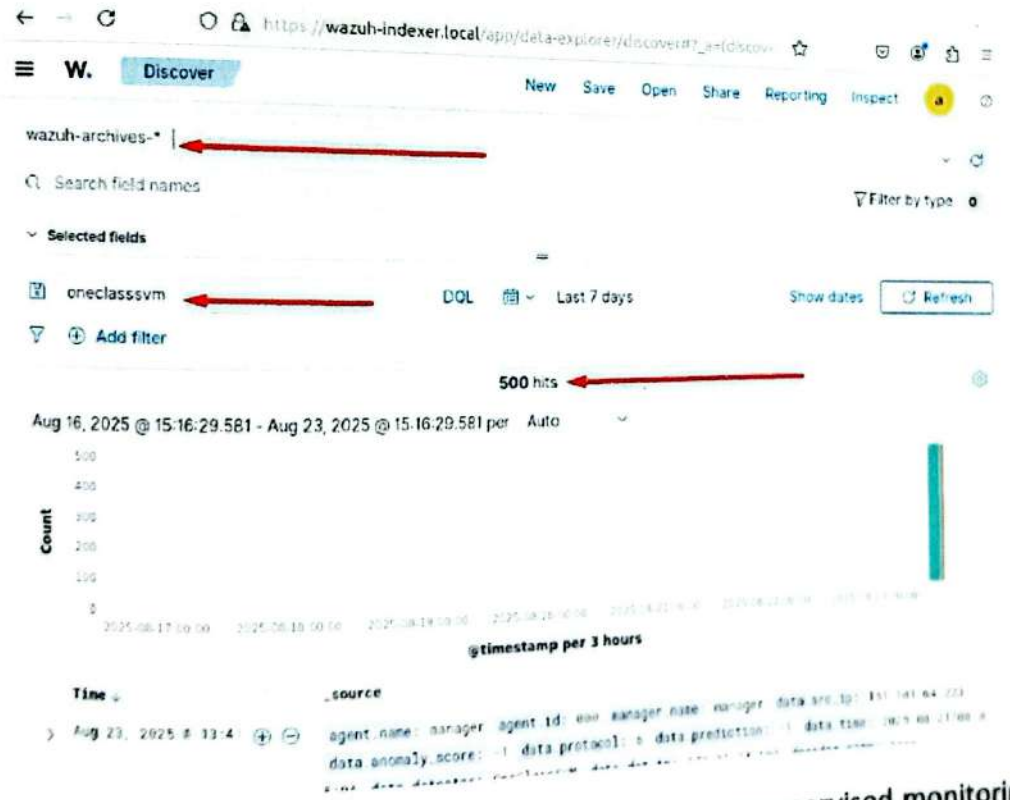

Figure 4.1.2 (b): One-Class SVM logs in achieve while unsupervised monitoring.

Ultimately, this difference underscores the importance of algorithm selection and parameter tuning in anomaly detection tasks. While One-Class SVM is effective when the concept of "normal" is well-defined and stable, it can fail to detect anomalies in dynamic environments or when the deviations are subtle. Isolation Forest, by comparison, may provide a more practical balance in detecting diverse anomaly types.

4.2 Attack Simulation and Alert Generation

4.2.1 YARA-based Malware Attack Simulation

In section 3.2 we have presented in details how YARA can be used to integrate with Wazuh agents to detect malware and suspicious files via rule based scans. This is accomplished through the enabling of the YARA module on the agent and application of custom or community YARA rules to the rules directory on the agent. When activated, the agent keeps scanning of files and directories depending on the configuration. When a file is identified, as matching one of the YARA rules an alert is generated containing the rule name, file path and matched strings. Then these alerts are sent towards the Wazuh manager in which they are processed and correlated with other security events to provide entire visibility of the threat.

A number of malware testing experiments were made in this project as they were carried out on the Linux agents, Windows agents, and, directly on the Wazuh manager. The findings have indicated that the Wazuh agent can detect suspicious behaviour and send alerts to the various entities promptly by employing the YARA rules that are incorporated therein. As an illustration, 4 malicious files had been detected during testing at time of download and alerts had been instantly generated to report the malicious discovery. The introduction of custom rules was found to be useful in detecting any suspicious activity in the agent as shown in Figure 4.2.1. These results verify the integrity and validity of integration, which supports the idea of Wazuh performing to integrate signature-based malware identification with a broader monitoring of behavior to improve the accuracy of the detection and resolve of threats.

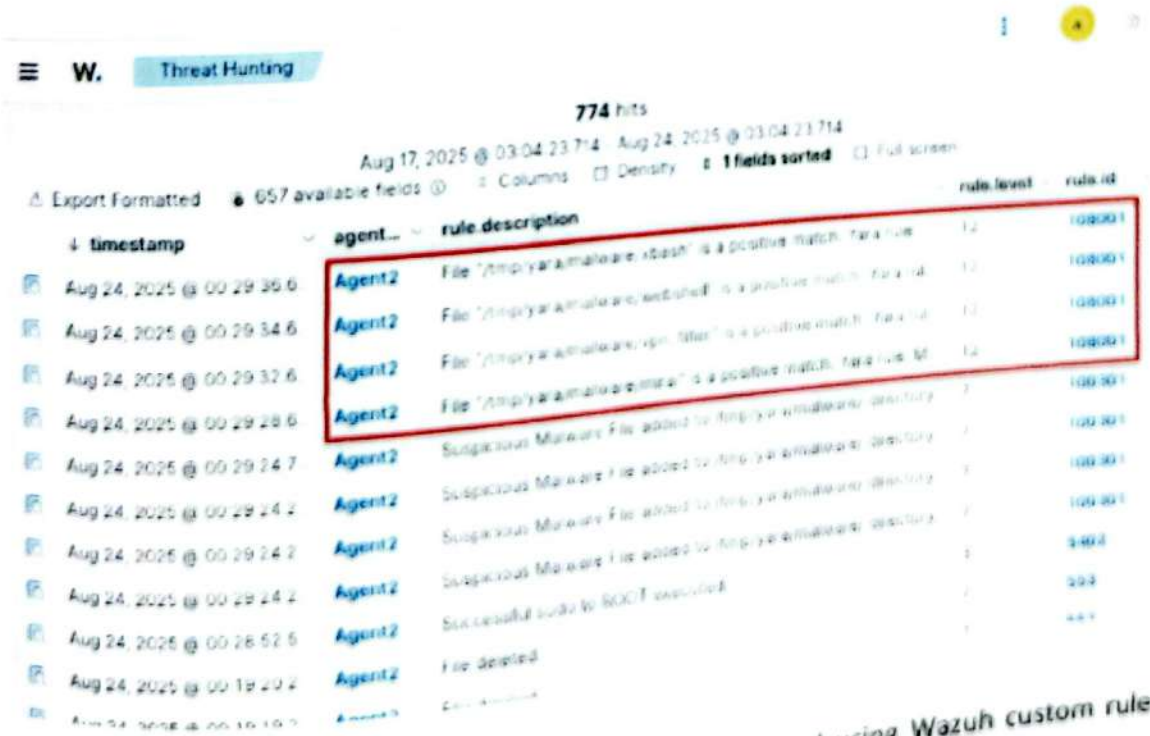

Figure 4.2.1: Malware file downloaded and alert generated using Wazuh custom rules.

4.2.2 Results From Isolation Forest During Attack Phase

During the live packet capturing using tcpdump, the extracted features were continuously updated in a CSV file via Python. Each new entry was then fed into the anomaly detection models—Isolation Forest and One-Class SVM—in real time. Once the models produced anomaly scores, the results were combined with the extracted features and converted into a JSON log format. These logs were subsequently forwarded by the Wazuh agent to the Wazuh manager, where they could be decoded, indexed, and visualized. The entire process was executed on the Wazuh agent, with results plotted in Jupyter Notebook. We lauched a YARA-based malware attack and downlaoded four malicious files in the /tmp/yara directory of the agent. Wazuh successfully detected and generated alerts for the malware, which were visible in the dashboard. However, the plotted outputs from the Isolation Forest and One-Class SVM models clearly reflect this malicious activity but not in the general process.

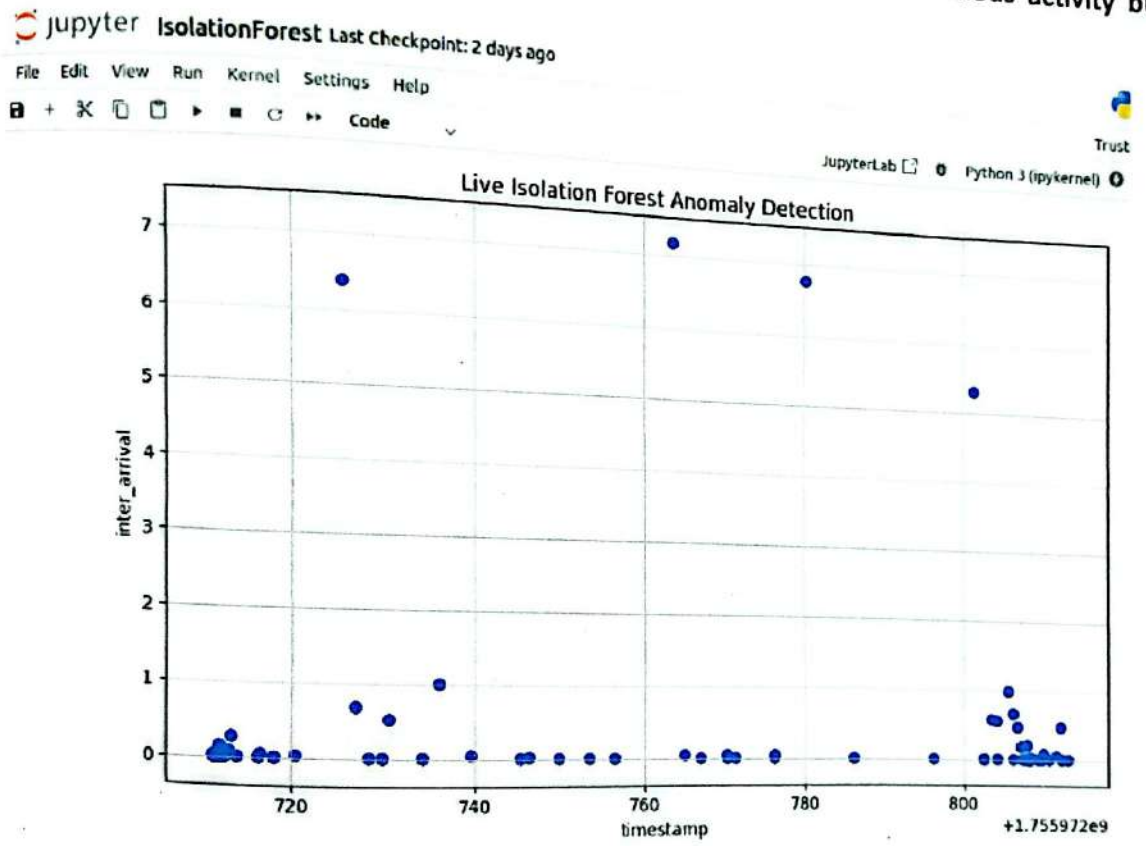

Figure 4.2.2 (a): Malware detected using YARA and Wazuh custom rules.

Although the scatter plot displays all points in the same color, this does not undermine the detection capability of the system. The uniform color representation is a result of how the visualization code was structured, where a separate color distinction between normal and anomalous points was not explicitly defined. However, the spatial distribution of the points in relation to the timeline still provides meaningful insights. The data points that appear farther away from the main cluster correspond to specific time intervals dur- ing which malware files were intentionally downloaded for testing. These deviations in placement clearly indicate anomalous behavior, demonstrating that the anomaly detec- tion mechanism successfully identified unusual events, even if the visualization does not differentiate them by color.

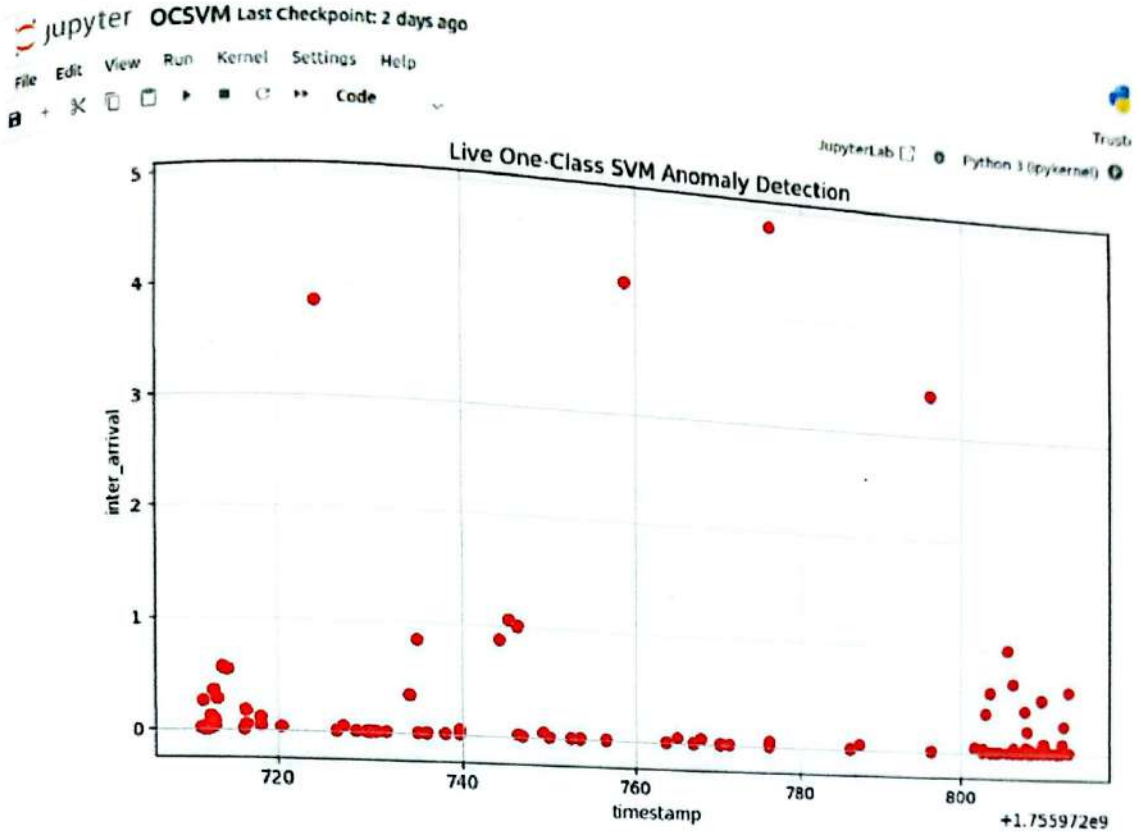

Figure 4.2.2 (b): OCSVM Malware detected using YARA and Wazuh custom rules.

4.2.3 Results From One-Class SVM During Attack Phase

In the case of the One-Class SVM visualization, the scatter plot similarly presents all points in a single color due to the way the plotting code was structured, without assigning separate visual markers for normal and anomalous data. Nevertheless, the distribution of points provides clear indications of anomalous activity. In particular, we observed distinct data points positioned noticeably farther from the regular cluster of points, mirroring the behavior observed with the Isolation Forest model. Importantly, the timestamps at which these outlying points appear correspond exactly to the time when the four malware files were downloaded to the /tmp/yara directory during testing. This temporal alignment strongly reinforces that the One-Class SVM model successfully detected deviations in network behavior at the precise moments when malicious activity occurred.

Although the absence of color distinction in the plot limits immediate visual separation of normal and anomalous events, the placement of the data points in relation to time clearly validates the model's anomaly detection capability. When correlated with the alerts generated by Wazuh through YARA-based malware detection, the results demonstrate that our project not only captured the malicious event but also confirmed it through the anomaly detection models. This highlights the effectiveness of integrating machine learning techniques with Wazuh to strengthen real-time threat detection.

4.3 Custom Dashboard

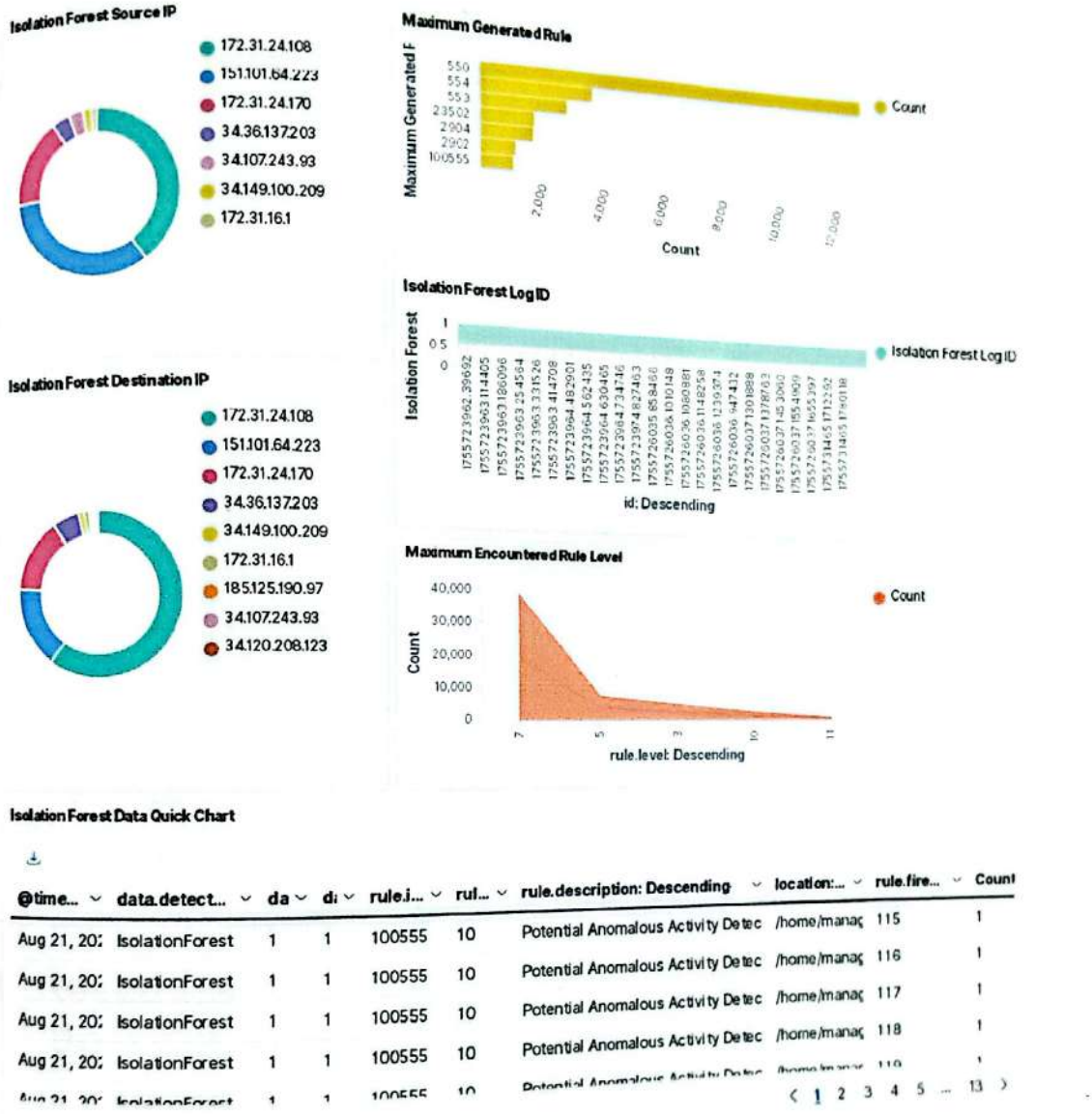

Figure 4.3: Wazuh custom dashboard with multiple features.

We will be able to customize the dashboard to follow alerts and as well as anomaly detection logs in a variety of formats created by Wazuh. A panel shows the source and destination IPs of events reported by the Isolation Forest model on a chart generated in Python. A horizontal dashboard shows the rule IDs that have been triggered the most, where Rule ID 550 is the most hit this time. Another panel is a custom panel that shows Isolation Forest logs with their unique identifiers and a curved indicator (in red) indicating the maximum rule severity level that is 7 in the present case. Moreover, a tabular dashboard will summarise the most critical information, such as the detection model, anomaly score, prediction, rule ID, rule level, rule description, location, and the timestamp at which the rule was activated and give a complete and effective understanding of what is going on in the system with anomalies.

4.4 Comparison with Other Platforms

We have contrasted our project with other platforms already in exist- ence of anomaly detection. Both Wazuh OpenSearch Anomaly Detection through Random Cut Forest and Elastic Security although close related to our project are however not the same. we have done differences in comparison of various features, flx- ibitly, scalability and other various facts and are found to stand out the best as far as performance goes in our current project.

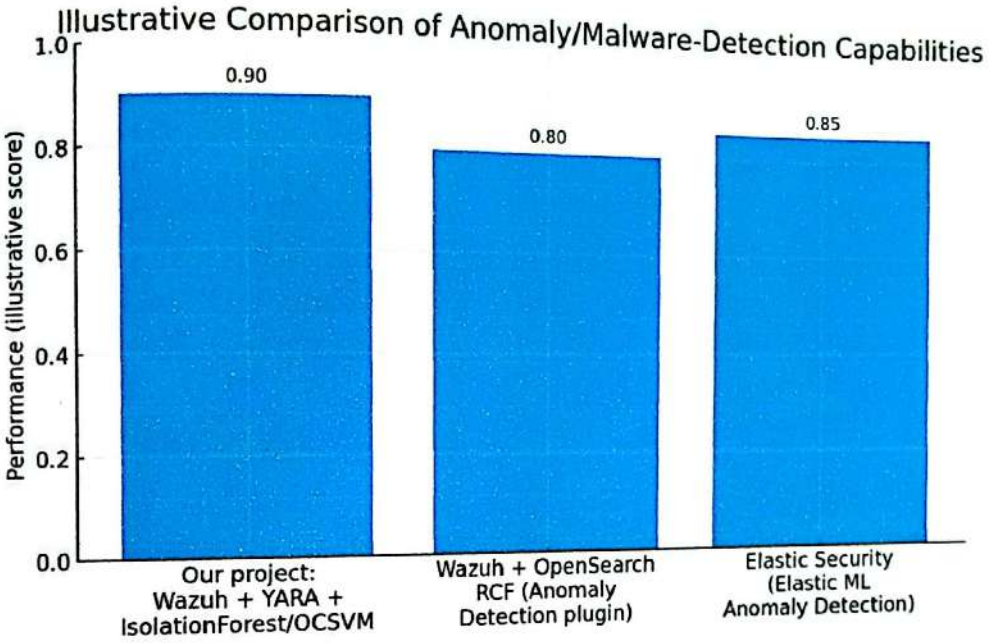

Figure 4.4: Comparing our project's performance with different platforms.

4.4.1 Wazuh with OpenSearch Anomaly Detection Using Random Cut Forest

Wazuh is integrated with the OpenSearch platform, which is a fork of the Elasticsearch. Anomaly Detection (AD) plugin is one of the enhanced Odessa capabilities of OpenSearch based on the Random Cut Forest (RCF) algorithm that automatically detects data points that are outside of the normal behavior. Combined with Wazuh, Wazuh security logs gathered by Wazuh agents and poured to the OpenSearch indexer are able to be processed in real-time by the AD plugin. This enables the SOC analysts to highlight abnormal patterns of activity, e.g. traffic spikes, suspicious logins, or strange system calls without having to set up specific signatures in advance. The strength of this system is its scalability and automation: OpenSearch is capable of ingesting a very large quantity of logs, and RCF im- proves on a statistically grounded scoring of anomalies. Nonetheless, the approach is highly unsupervised and black-box in nature and so the detection parameters are prone to false positive hits requiring manual optimization to achieve satisfactory results. This solution is also familiar to the organizations already implementing the Wazuh and OpenSearch stack because it does not require an external ML pipeline.

4.4.2 Elastic Security (Elastic ML Anomaly Detection)

Elastic Security is an extension to the Elastic Stack (Elasticsearch, Logstash, Kibana, Beats) and it provides in-built Security Information and Events Management (SIEM) facility. Among its high abilities is the Machine Learning Anomaly Detection, where the user can create unsupervised anomaly detection jobs on ingested data like authentication logs, network flows or process event data. Elastic ML jobs apply statistical and ML-based models to discover normal baseline behavior and use that knowledge to assign anomaly scores when deviations are detected. This comes in handy in user behavior analytics (UBA) and entity behavior analytics (UEBA), where unusual patterns in login times, geographic access or aberrant process executions play a major role. In comparison to the built-in ML capabilities of Wazuh, Elastic Security will offer a more complete ML interface, with rich visualization in Kibana, automation of the whole job creation approach, as well as enabling the interpretation of results as anomalies. Elastic Security is highly used in enterprises since it is user-friendly as well as supported commercially and is easy to integrate into Elastic-based systems and platforms, although its superior ML functionality is usually provided on a paid basis.

4.4.3 Why Our Project Stands Out

Unlike the existing systems, our project introduces a hybrid threat detection approach by combining YARA-based malware detection with machine learning–based network anomaly detection (Isolation Forest and One-Class SVM) inside the Wazuh SIEM ecosys- tem. This dual-layer strategy ensures that both known threats (e.g., malicious files, malware signatures) and unknown or zero-day anomalies (e.g., abnormal network traffic patterns) are detected in real-time. While Wazuh + OpenSearch RCF focuses primarily on unsupervised anomaly scoring and Elastic Security emphasizes enterprise-level ML features, our framework directly integrates with the Wazuh manager and dashboard, making detection results immediately visible to SOC analysts without requiring additional plugins or licenses. Moreover, the project enhances Wazuh with custom rules, decoders, and a tailored dashboard for visualization, which allows greater flexibility and control compared to the more generic anomaly detection interfaces in Elastic or OpenSearch. This integration makes our system lightweight, customizable, and cost-effective, while still providing strong detection accuracy across multiple platforms (Linux, Windows, and Wazuh Manager itself), which many other solutions don't combine in one pipeline.

Chapter 5

Challenges & Future Works

5.1 Limitations

Project Limitations:

1. **Hardware limits:** All the project was designed on a single laptop using 32GB Ram and 200GB physical space and this limited the project resources significantly. There was a limit of 8 GB RAM to every virtual machine (VM) and a storage capacity of each VM was restricted. Such hardware limitations had an impact on scalability and the performance of the system.

2. **Wazuh Server Limitations:** The theoretical situation is that both Wazuh servers (master and worker) should be running concurrently. Nevertheless, there was not enough RAM to have more than one server running at a similar time. As a result there were limited testing and evaluation of a fully distributed environment.

3. **Limitation in operation of the agents:** It was not possible to run both agents simultaneously. The laptop did not perform well to cope with the load and system kept on hanging. Thus, the experiments were done on a single agent at each time. Simultaneously switching between the working segments was also restricted due to the need to shut down the Windows agent when Linux agent was in use and vice versa.

4. **Duration Limitation of monitoring:** Duration limitation of network monitoring through some of the tools like TCPDump and Wireshark was limited. The sample captures were variable in terms of time interval with some taking up to 5-6 hours and others less than one hour. Consequently, the training and modeling data based on the Isolation Forest and One-Class SVM were preconditioned by stationary and finite time intervals, which possibly distorted the accuracy and generalizing of anomaly detection models.

5. **Attack Simulation Limitations**: The original design specification called to simulate brute-force and DDoS type requests to test the system under those conditions; however, the current hardware constraints would not permit this to be feasible. Thus, only YARA-based malware detection was studied in the project.

5.2 Challenges

Project Challenges

- **Network Configuration and Flexibility:** Our original distributed multi-node Wazuh architecture was at Hyper-V and used fixed IP addresses on the virtual switch. Although effective, this was a fixed-router or network-based setting that lacked flexibility. The project needed to be flexible and able to function in any network therefore necessitating a switch to DHCP. This change offered major challenges, in that regeneration of certificates is needed, and use of DNS names (e.g., wazuh-indexer.local, wazuh-manager.local, wazuh-dashboard.local) rather than fixed IP addresses is the solution.

- **Resource Limitations in Virtualized Environment:** The next major challenge was running all Wazuh components on the same laptop, which had restriction- ed RAM and disk space. The maximum number of VMs allowed to run simultaneously was three (usually, the Wazuh Indexer, one Wazuh Manager node (master or worker), and a single agent (Linux or Windows)). Setting up the smooth running without overloading the host machine or making any VM hang needed a continuous optimization process and resource balance.

- **Live Anomaly Detection Workflow:** The most challenging area was the implementation of a full workflow of live anomaly detection. This involved live measurements of the data on the network, the extraction of features relevant to it, feeding these features to both Isolation Forest and One-Class SVM models, returning anomaly scores in real-time and storing them as structured JSON logs. The computational and design requirements of executing this entire pipeline within Jupyter Notebook and also plotting and visualizing the results were challenging and quite demanding on the computing resources.

- **Integration with Wazuh through Custom Rules:** The other basic difficulty was the development of custom Wazuh rules to integrate the anomaly detection models. Exporting the results of Isolation Forest and One-Class SVM to Wazuh alerts and archived logs needed extensive proficiency in the Wazuh rules, decoders, and alerting engine. This was important to identify anomalies that were identified by the models to be reported and viewed within SIEM environment.

5.3 Future Work

Evaluation Under Diverse Attack Scenarios:

So far, the system was applied to testing on malware detection with YARA rules. In future research, we will assess the framework against a broader catalog of possible attacks to include brute-force logins and Distributed Denial-of-Service (DDoS) attacks. By examining the way the system reacts to such attacks, more information will be gained about the robustness and scalability of the system and how it can detect various types of threats.

Application in the Real-world:

The present solution was tested on a controlled virtualization environment One of the most important steps that should be undertaken is a real production testbed. Under real-life conditions, running the system at scale will enable us to assess its performance, stability, and scaling as well as gain some practical experience in terms of the challenges of managing log volume, prioritising alerts, and other issues concerning the resiliency of the system.

Improvement of Feature engineering:

Currently, the anomaly detection framework utilizes a small range of fundamental features, such as timestamp, source and destination IP addresses, ports and protocol type. Although helpful, these features fail to encompass all the intricacies of the current cyber threats. Future effort will be devoted to the enhancement of the feature set to include more advanced traffic properties, i.e., packet size distributions, flow duration, inter-arrival times, byte and packet counts, directionality of connections, and flag patterns in TCP. In addition, features involving statistics (e.g., mean, variance and standard deviation of packet sizes) and entropy measures of randomness of the traffic will also be considered. By increasing the features space, models will be more accurate in differentiating benign and malicious behaviours, minimising false positives, and responding to a wider (possibly more generalised to a broader set of attack vectors).

Chapter 6

Conclusion

Our project has managed to verify the effectiveness of an integrated and adaptive model of threat detection that can solve the serious limitations of the more traditional and static measures in the cybersecurity domain. The methodology consisted in setting up a multinode Wazuh infrastructure taking advantage of its main components (Indexer, server and agents) to build an integrated Security Information and Event Management (SIEM) platform. This platform played a key role allowing us to see and monitor security data in real-time.

The overall novelty of the framework is that it successfully integrates YARA-based file scanning with unsupervised machine learning techniques such as Isolation Forest and One-Class SVM in the form of the dual-layered defense mechanism. Our attack simulations proved the practicability of the framework. We have been able to identify four malicious files using the YARA rules, and the machine learning models give vital confirmation at providing clear anomalies on the network traffic at the time of the attacks. This has proven that our system can not only detect known threats but also detect unknown network anomalies further augmenting threats detection capabilities.

Some difficulties and flaws were experienced in this study. The most important one was the limitation on the side of hardware, as it did not allow us to perform more comprehensive and sophisticated simulations of attacks, like the brute-force attacks or the DDoS ones. This constraint ruled out our capacity to comprehensively exercise the robustness and scalability of the framework to workloads of higher levels.

To a future research, we would suggest that future work can be undertaken to improve on some of the hardware limitations to be able to simulate more advanced attacks. This would give a better analysis on the performance and scalability of the framework. Moreover, the system might be developed to include more powerful machine learning models or to identify the sources of new data that would help the system to become even more powerful at the detection of threats. The literature that we have been referring to throughout this paper, including scholarly articles as well as technical literature, was an essential contribution to the design of this powerful framework.

Appendix A

Additional Codes

Wazuh-YARA Active Response Scanning Code

```
# !/bin/bash
# Wazuh - Yara active response
# Copyright (C) 2015-2022, Wazuh Inc.
#
# This program is free software; you can redistribute it
# and/or modify it under the terms of the GNU General Public
# License (version 2) as published by the FSF - Free Software
# Foundation.


#------------------ Gather parameters ------------------#

# Extra arguments
read INPUT_JSON
YARA_PATH=$(echo $INPUT_JSON | jq -r .parameters.extra_args[1])
YARA_RULES=$(echo $INPUT_JSON | jq -r .parameters.extra_args[3])
FILENAME=$(echo $INPUT_JSON | jq -r .parameters.alert.syscheck.path)

# Set LOG_FILE path
LOG_FILE="logs/active-responses.log"

size=0
actual_size=$(stat -c %s ${FILENAME})
while [ ${size} -ne ${actual_size} ]; do
    sleep 1
    size=${actual_size}
    actual_size=$(stat -c %s ${FILENAME})
done

#------------------ Analyze parameters ------------------#

if [[ ! $YARA_PATH ]] || [[ ! $YARA_RULES ]]
then
```

```
    echo "wazuh-yara: ERROR - Yara active response error.
    Yara path and rules parameters are mandatory." >> ${LOG_FILE}
    exit 1
fi

#---------- Main workflow ----------#

# Execute Yara scan on the specified filename
yara_output="$("${YARA_PATH}"/yara -w -r "$YARA_RULES" "$FILENAME")"

if [[ $yara_output != "" ]]
then
    # Iterate every detected rule and append it to the LOG_FILE
    while read -r line; do
        echo "wazuh-yara: INFO - Scan result: $line" >> ${LOG_FILE}
    done <<< "$yara_output"
fi
exit 0;
```

Feature Extraction Code

```
import os
import time
import pandas as pd
from scapy.all import PcapReader, TCP, UDP, IP
from collections import deque

PCAP_FILE = "capture.pcap"
CSV_FILE = "output.csv"
MAX_FILE_SIZE = 50 * 1024 * 1024  # 50 MB
WINDOW_SIZE = 100  # sliding window of last 100 packets

# Rolling buffer of packets
packet_buffer = deque(maxlen=WINDOW_SIZE)
last_timestamp = None

def extract_features(pkt):
    global last_timestamp
    features = {}

    if IP not in pkt:
        return None  # skip non-IP packets

    ip = pkt[IP]
    features["timestamp"] = pkt.time
```

```
    # inter-arrival time
    if last_timestamp is not None:
        features["inter_arrival"] = pkt.time - last_timestamp
    else:
        features["inter_arrival"] = 0
    last_timestamp = pkt.time

    features["src_ip"] = ip.src
    features["dst_ip"] = ip.dst
    features["protocol"] = ip.proto
    features["length"] = len(pkt)

    #TCP/UDP ports and flags
    if ip.proto == 6 and TCP in pkt:
        tcp = pkt[TCP]
        features["srcport"] = tcp.sport
        features["dst_port"] = tcp.dport
        features["flags"] = str(tcp.flags)
    elif ip.proto == 17 and UDP in pkt:
        udp = pkt[UDP]
        features["srcport"] = udp.sport
        features["dstport"] = udp.dport
        features["flags"] = "NONE"
    else:
        features["srcport"] = None
        features["dstport"] = None
        features["flags"] = "NONE"

    return features

def compute_window_features():
    """Compute aggregated features over sliding window."""
    if not packet_buffer:
        return {}

    df = pd.DataFrame(packet_buffer)
    window_features = {
        "pkt_rate": len(df) / (df["inter_arrival"]
        .sum() + 1e-6),  # packets per sec
        "byte_rate": df["length"].
        sum() / (df["inter_arrival"].sum() + 1e-6),
        # bytes per sec
        "unique_src_ports": df["src_port"].nunique(),
        "unique_dst_ports": df["dst_port"].nunique(),
    }
    return window_features
```

```
def write_to_csv_top(features):
    """Prepend the latest features to CSV, keep max size
    50MB."""
    new_row = pd.DataFrame([features])

    if os.path.exists(CSV_FILE):
        # Read existing CSV
        df_existing = pd.read_csv(CSV_FILE)
        # Prepend new row
        df_combined = pd.concat([new_row, df_existing],
        ignore_index=True)
    else:
        df_combined = new_row

    # Truncate if file exceeds MAX_FILE_SIZE
    temp_file = "temp.csv"
    df_combined.to_csv(temp_file, index=False)
    while os.path.getsize(temp_file) > MAX_FILE_SIZE and
    len(df_combined) > 1:
        # Remove the last (oldest) row
        df_combined = df_combined.iloc[:-1]
        df_combined.to_csv(temp_file, index=False)

    os.replace(temp_file, CSV_FILE)

def live_process():
    print(f"[+] Monitoring {PCAP_FILE} and writing live
    features to {CSV_FILE} ...")
    with PcapReader(PCAP_FILE) as pcap:
        for pkt in pcap:
            features = extract_features(pkt)
            if features:
                # Add packet to rolling buffer
                packet_buffer.append(features)

                # Compute sliding window features
                window_features = compute_window_features()

                # Merge base + window features
                features.update(window_features)

                # Write latest row to top of CSV
                write_to_csv_top(features)

if __name__ == "__main__":
    while True:
        try:
            live_process()
```

```
        except Exception as e:
            print("Waiting for new packets...", str(e))
            time.sleep(2)
```

Isolation Forest Code

```
import pandas as pd
from sklearn.ensemble import IsolationForest
import matplotlib.pyplot as plt
from IPython.display import clear_output, display
from collections import deque
import os

OUTPUT_CSV = '/home/master/tcpDumpLive/output.csv'
ANOMALY_CSV = '/home/master/tcpDumpLive/IsolationForestLive/
anomalies.csv'
ROLLING_SIZE = 500  # Maximum rows in anomalies.csv

plt.ion()
fig, ax = plt.subplots(figsize=(10,6))

# Rolling deque for anomalies
anomaly_queue = deque(maxlen=ROLLING_SIZE)

while True:
    if os.path.exists(OUTPUT_CSV):
        try:
            # Read only the first row of output.csv
            row = pd.read_csv(OUTPUT_CSV, nrows=1)
            X_row = row.select_dtypes(include=['float64',
            'int64']).dropna()
            if X_row.empty:
                continue

            # Fit IsolationForest on this single row
            model = IsolationForest(contamination=0.05,
            random_state=42)
            model.fit(X_row)
            row['anomaly'] = model.predict(X_row)

            # Append to rolling anomaly queue
            anomaly_queue.append(row.iloc[0])

            # Convert deque to DataFrame for saving and
            visualization
```

```
            df_anomaly = pd.DataFrame(anomaly_queue)
            df_anomaly.to_csv(ANOMALY_CSV, index=False)

            # Live visualization
            X_vis = df_anomaly.select_dtypes(include=['float64',
            'int64']).
            colors = df_anomaly['anomaly'].map({1: 'blue', -1:
            'red'})

            clear_output(wait=True)
            ax.clear()
            if X_vis.shape[1] >= 2:
                ax.scatter(X_vis.iloc[:,0], X_vis.iloc[:,1],
                c=colors, marker='o')
                ax.set_xlabel(X_vis.columns[0])
                ax.set_ylabel(X_vis.columns[1])
            else:
                ax.scatter(range(len(X_vis)), X_vis.iloc[:,0],
                c=colors, marker='o')
                ax.set_xlabel('Index')
                ax.set_ylabel(X_vis.columns[0])
            ax.set_title('Live IsolationForest Anomaly Detection')
            ax.grid(True)
            display(fig)

            # Display latest 5 anomalies in notebook
            display(df_anomaly.tail(5))

        except pd.errors.EmptyDataError:
            # Skip if output.csv is empty temporarily
            continue
    else:
        print(f"{OUTPUT_CSV} not found. Waiting for it...")
```

One-Class SVM Code

```
import pandas as pd
from sklearn.svm import OneClassSVM
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
from IPython.display import clear_output, display
from collections import deque
import os

# Paths
```

```
OUTPUT_CSV = '/home/master/tcpDumpLive/output.csv'
ANOMALY_CSV = '/home/master/tcpDumpLive/OneClassSVMLive/
oneclass_anomalies.csv'
ROLLING_SIZE = 500  # Maximum rows in anomalies.csv

# Live plot setup
plt.ion()
fig, ax = plt.subplots(figsize=(10,6))

# Rolling deque for anomalies
anomaly_queue = deque(maxlen=ROLLING_SIZE)

while True:
    if os.path.exists(OUTPUT_CSV):
        try:
            # Read only the first row of output.csv
            row = pd.read_csv(OUTPUT_CSV, nrows=1)
            X_row = row.select_dtypes(include=['float64',
            'int64']).dropna()
            if X_row.empty:
                continue

            # Scale the data (important for One-Class SVM)
            scaler = StandardScaler()
            X_scaled = scaler.fit_transform(X_row)

            # Fit One-Class SVM on this single row
            model = OneClassSVM(kernel='rbf', gamma='auto',
            nu=0.05)
            model.fit(X_scaled)
            row['anomaly'] = model.predict(X_scaled)

            # Append to rolling anomaly queue
            anomaly_queue.append(row.iloc[0])

            # Convert deque to DataFrame for saving and
            visualization
            df_anomaly = pd.DataFrame(anomaly_queue)
            df_anomaly.to_csv(ANOMALY_CSV, index=False)

            # Live visualization
            X_vis = df_anomaly.select_dtypes(include=['float64',
            'int64'])
            colors = df_anomaly['anomaly'].map({1: 'blue', -1:
            'red'})

            clear_output(wait=True)
            ax.clear()
```

```
            if X_vis.shape[1] >= 2:
                ax.scatter(X_vis.iloc[:,0], X_vis.iloc[:,1],
                c=colors, marker='o')
                ax.set_xlabel(X_vis.columns[0])
                ax.set_ylabel(X_vis.columns[1])
            else:
                ax.scatter(range(len(X_vis)), X_vis.iloc[:,0],
                c=colors, marker='o')
                ax.set_xlabel('Index')
                ax.set_ylabel(X_vis.columns[0])
            ax.set_title('Live One-Class SVM Anomaly Detection')
            ax.grid(True)
            display(fig)

            # Display latest 5 anomalies in notebook
            display(df_anomaly.tail(5))

        except pd.errors.EmptyDataError:
            # Skip if output.csv is empty temporarily
            continue
    else:
        print(f"{OUTPUT_CSV} not found. Waiting for it...")
```

Anomaly Scoring form Isolation Forest and Json log CreationCode

```
    # iso_to_wazuh.py
import os
import json
import time
from datetime import datetime
from collections import deque
import pandas as pd

# ==== CONFIG ====
CSV_FILE = "anomalies.csv"          # Isolation Forest CSV (live)
LOG_FILE = "anomaly_if.log"         # Output JSON log for Wazuh
MAX_LOG_LINES = 500                 # Rolling limit

DETECTOR_NAME = "IsolationForest"
# ================

# Rolling buffer (survives restarts by loading existing log, if any)
log_buffer = deque(maxlen=MAX_LOG_LINES)
if os.path.exists(LOG_FILE):
    with open(LOG_FILE, "r") as f:
```

```
        for line in f:
            line = line.strip()
            if line:
                log_buffer.append(json.loads(line))

# Track how many rows we have already exported from the CSV
last_rows = 0

def _first_non_null(row, keys, default="N/A"):
    for k in keys:
        if k in row and pd.notna(row[k]):
            return row[k]
    return default

def _int_from(row, keys, default=1):
    for k in keys:
        if k in row and pd.notna(row[k]):
            try:
                return int(row[k])
            except Exception:
                pass
    return default

def _float_from(row, keys, default=None):
    for k in keys:
        if k in row and pd.notna(row[k]):
            try:
                return float(row[k])
            except Exception:
                pass
    return default

def build_event(row_dict):
    # Required fields (with fallbacks)
    src_ip   = _first_non_null(row_dict, ["src_ip",
    "source ip"])
    dst_ip   = _first_non_null(row_dict, ["dst_ip",
    "destination ip"])
    protocol =_first_non_null(row_dict, ["protocol",
    "proto"])

    # prediction: prefer explicit 'prediction'; else
    'anomaly'; else default 1
    prediction =_int_from(row_dict, ["prediction",
    "anomaly", "if_pred"], default=1)

    # anomaly score: prefer explicit; else derive -1 for
    anomalies, 1 for normal
```

```
    score = _float_from(row_dict, ["anomaly_score", "score",
    "if_score"],
        default=(-1.0 if prediction == -1 else 1.0))

    event = {
        "timestamp": datetime.utcnow().strftime("%Y-%m-%dT%H:%M:%S"),
        "src_ip": src_ip,
        "dst_ip": dst_ip,
        "protocol": protocol,
        "detector": DETECTOR_NAME,
        "anomaly_score": score,
        "prediction": prediction
    }
    return event

def flush_log():
    with open(LOG_FILE, "w") as f:
        for e in log_buffer:
            f.write(json.dumps(e) + "\n")

print(f" Watching {CSV_FILE} to {LOG_FILE} (max {MAX_LOG_LINES} lines)")
while True:
    try:
        if not os.path.exists(CSV_FILE):

            continue

        df = pd.read_csv(CSV_FILE)
        # If the CSV was rotated/truncated, reset our pointer
        if len(df) < last_rows:
            last_rows = 0

        if len(df) > last_rows:
            new = df.iloc[last_rows:].to_dict(orient="records")
            for row in new:
                event = build_event(row)
                log_buffer.append(event)
            flush_log()
            last_rows = len(df)

    except Exception as e:
        # Non-fatal; keep monitoring
        # print("ISO pipeline error:", e)
        pass
```

Anomaly Scoring form One-Class SVM and JSON log Creation Code

```
    # ocsvm_to_wazuh.py
import os
import json
import time
from datetime import datetime
from collections import deque
import pandas as pd

# ==== CONFIG ====
CSV_FILE = "ocsvm_anomalies.csv"          # OCSVM CSV (live)
LOG_FILE = "anomaly_ocsvm.log"            # Output JSON log for Wazuh
MAX_LOG_LINES = 500                       # Rolling limit

DETECTOR_NAME = "OneClassSVM"
# ================

# Rolling buffer (survives restarts by loading existing log, if any)
log_buffer = deque(maxlen=MAX_LOG_LINES)
if os.path.exists(LOG_FILE):
    with open(LOG_FILE, "r") as f:
        for line in f:
            line = line.strip()
            if line:
                log_buffer.append(json.loads(line))

# Track how many rows we have already exported from the CSV
last_rows = 0

def _first_non_null(row, keys, default="N/A"):
    for k in keys:
        if k in row and pd.notna(row[k]):
            return row[k]
    return default

def _int_from(row, keys, default=1):
    for k in keys:
        if k in row and pd.notna(row[k]):
            try:
                return int(row[k])
            except Exception:
                pass
    return default

def _float_from(row, keys, default=None):
    for k in keys:
        if k in row and pd.notna(row[k]):
```

```
            try:
                return float(row[k])
            except Exception:
                pass
    return default

def build_event(row_dict):
    # Required fields (with fallbacks)
    src_ip = _first_non_null(row_dict, ["src_ip",
    "source ip"])
    dst_ip = _first_non_null(row_dict, ["dst_ip",
    "destination ip"])
    protocol =_first_non_null(row_dict, ["protocol",
    "proto"])

    # prediction: prefer explicit 'prediction';
    else 'anomaly'; else 'ocsvm_pred'; else 1
    prediction =_int_from(row_dict, ["prediction", "anomaly",
    "ocsvm_pred"], default=1)

    # anomaly score: prefer explicit; else derive -1 for
    anomalies, 1 for normal
    score =_float_from(row_dict, ["anomaly_score", "score",
    "ocsvm_score"],
        default=(-1.0 if prediction == -1 else 1.0))

    event = {
        "timestamp": datetime.utcnow().strftime("%Y-%m-%dT%H:%M:%S"),
        "src_ip": src_ip,
        "dst_ip": dst_ip,
        "protocol": protocol,
        "detector": DETECTOR_NAME,
        "anomaly_score": score,
        "prediction": prediction
    }
    return event

def flush_log():
    with open(LOG_FILE, "w") as f:
        for e in log_buffer:
            f.write(json.dumps(e) + "\n")

# print(f"Watching {CSV_FILE} to {LOG_FILE} (max {MAX_LOG_LINES} lines)")
while True:
    try:
        if not os.path.exists(CSV_FILE):
            time.sleep(POLL_SEC)
            continue
```

```
            df = pd.read_csv(CSV_FILE)
            # If the CSV was rotated/truncated, reset our pointer
            if len(df) < last_rows:
                last_rows = 0

            if len(df) > last_rows:
                new = df.iloc[last_rows:].to_dict(orient="records")
                for row in new:
                    event = build_event(row)
                    log_buffer.append(event)
                flush_log()
                last_rows = len(df)

    except Exception as e:
        # Non-fatal; keep monitoring
        # print("OCSVM pipeline error:", e)
        pass
```

Appendix B

Additional Images

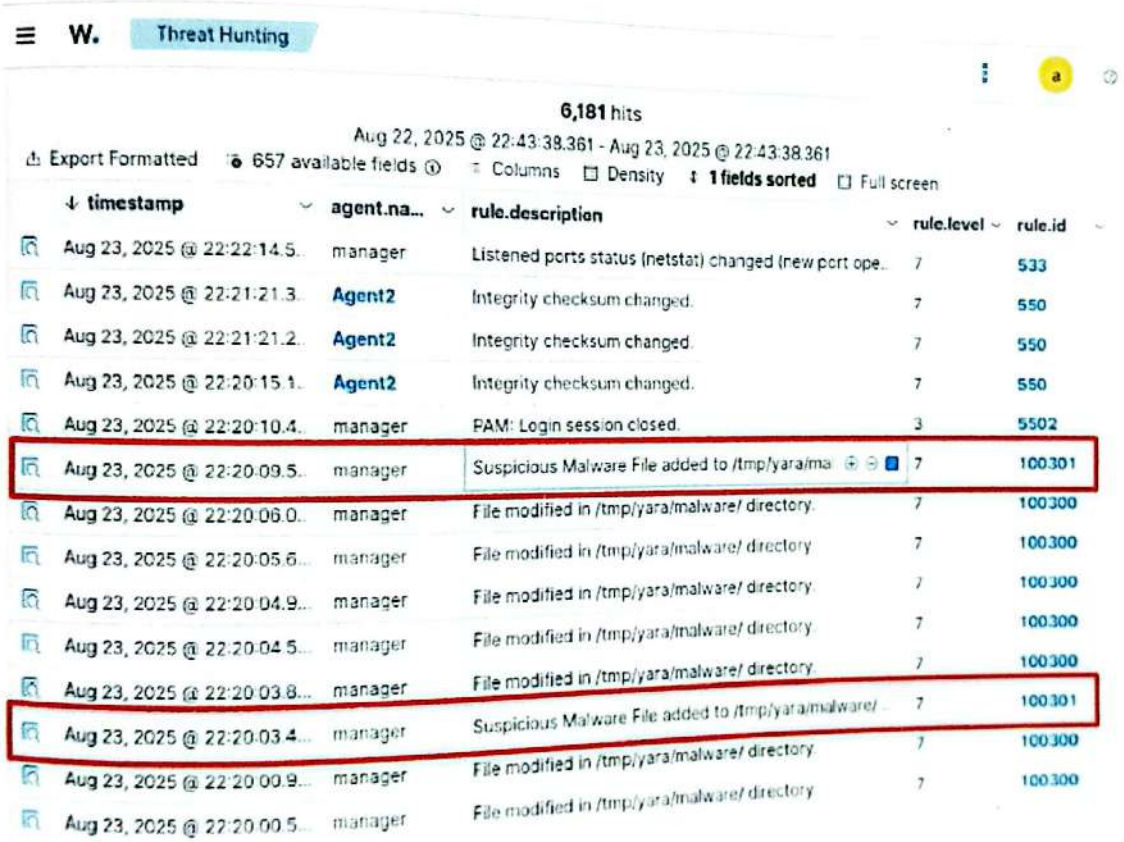

Figure B.1: Malware detected in Wazuh manager.

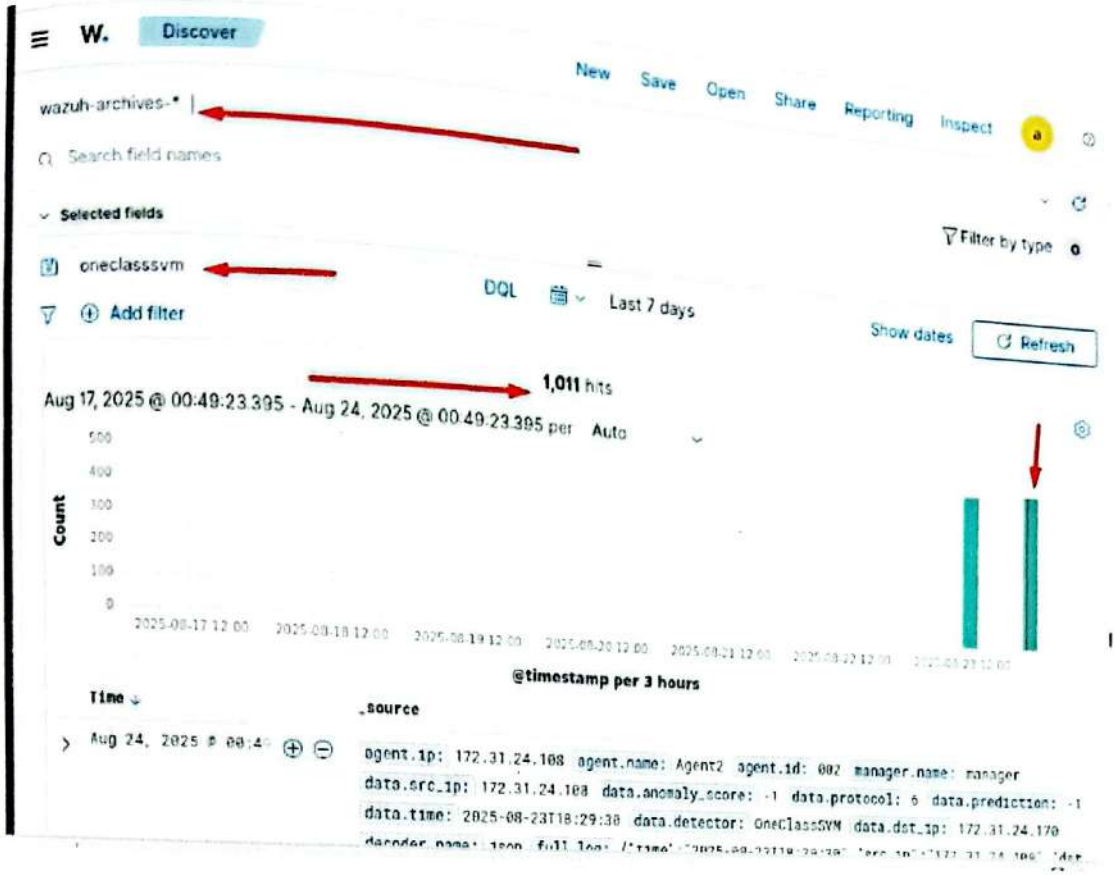

Figure B.2: More OneClassSVM logs collected from Wazuh agent.

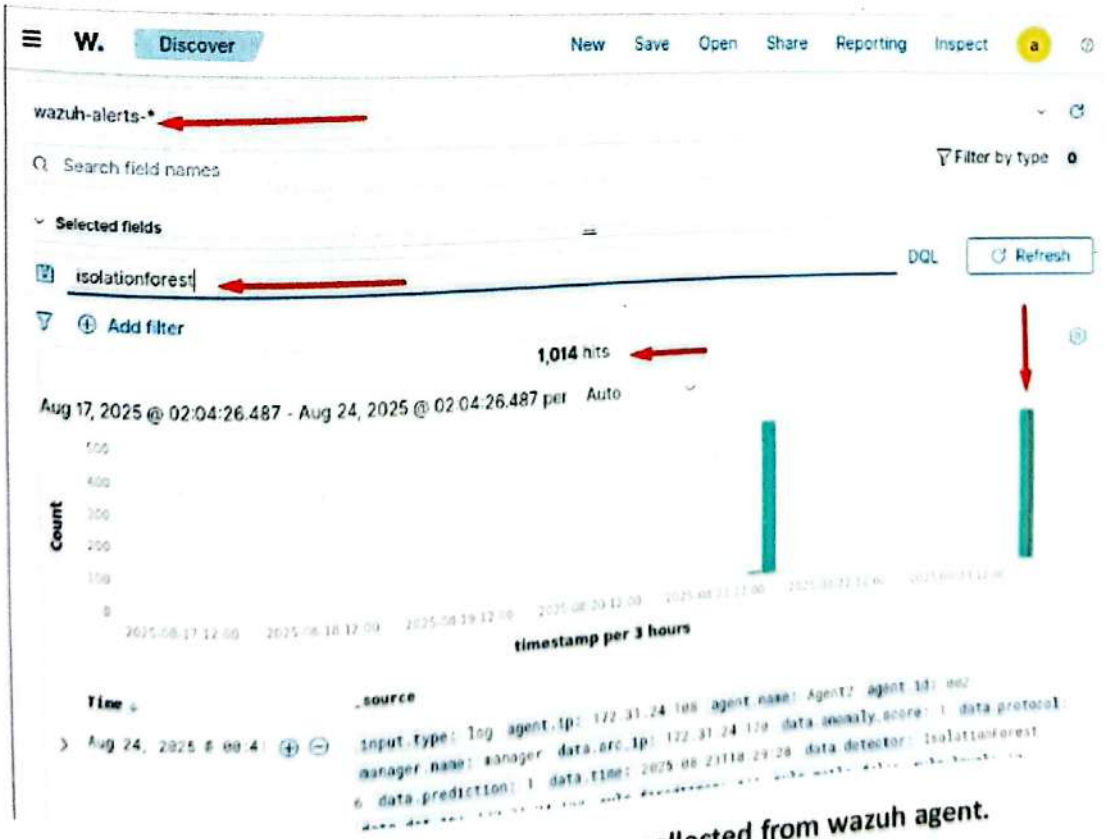

Figure B.3: More Isolation Forest log collected from wazuh agent.

Bibliography

[1] Wassim Ahmad. Unveiling anomalies: Leveraging machine learning for internal user behavior analysis – top 10 use cases. *Journal of Cybersecurity and Data Analytics*, XX(Y):PP–QQ, 2025.

[2] Wasan Saad Ahmed and Ziyad Tariq Mustafa Al-Ta'i. Analysis of wazuh siem's effectiveness in cloud security monitoring. *Journal of Cybersecurity and Information Management*, 15(1):244–250, 2023.

[3] Resma Tania Ghosh, Shitanshu Tiwari, Syarif Hidayatullah, Viky Herdika Sandi, Munawar Agus Riyadi, Alexander Pandra Mala Tarigan, Nanda Rifan Febrian, and Daday Hernandi Drajat. Implementation of security information & event management (siem) wazuh with active response and telegram notification for mitigating brute force attacks on the gt-i2ti usakti information system. *Intelmatics*, 4(1):1–7, March 2024.

[4] Siddhant Gupta, Fred Lu, Andrew Barlow, Edward Raff, Francis Ferraro, Cynthia Matuszek, Charles Nicholas, and James Holt. Living off the analyst: Harvesting features from yara rules for malware detection, 2024. CyberHunt 2024 / BigData'24 workshop paper.

[5] Amirhossein Hooshmand, Sanjay Huchaiah, Ali Alzighaibi, Tarek Hashim, Emad Atlam, and Abdelwahed M. Gad. Robust network anomaly detection using ensemble learning approach and explainable artificial intelligence (xai). *Alexandria Engineering Journal*, 94:120–130, 2024.

[6] Md Rafiqul Islam and Raisa Rafique. Wazuh siem for cyber security and threat mitigation in apparel industries. *International Journal of Engineering Materials and Manufacture*, 9(4):136–144, 2024.

[7] Shijia Li, Jiang Ming, Pengda Qiu, Qiyuan Chen, Lanqing Liu, Huaifeng Bao, Qiang Wang, and Chunfu Jia. Packgenome: Automatically generating robust yara rules for accurate malware packer detection. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 3078–3092, Copenhagen, Denmark, 2023. Association for Computing Machinery.

[8] Willian T. Lunardi, Martin Andreoni Lopez, and Jean-Pierre Giacalone. ARCADE: Adversarially regularized convolutional autoencoder for network anomaly detection, 2022.

[9] Ziad Mansour, Weihan Ou, Steven H. H. Ding, Mohammad Zulkernine, and Philippe Charland. Neuroyara: Learning to rank for YARA rules generation through deep

language modeling and discriminative n-gram encoding. *IEEE Transactions on Dependable and Secure Computing*, 22(2):1747–1762, 2025. Early Access 2024; final pagination in 2025.

[10] MDPI. Improving threat detection in wazuh using machine learning techniques. *Journal of Cybersecurity and Privacy*, 5(2):34, 2025. Please update the author list if needed.

[11] Fariha Moomtaheen, Sikha S. Bagui, Subhash C. Bagui, and Dustin Mink. Extended isolation forest for intrusion detection in zeek data. *Information*, 15(7):404, 2024.

[12] Edward Raff, Richard Zak, Gary Lopez Munoz, William Fleming, Hyrum S. Ander- son, Bobby Filar, Charles Nicholas, and James Holt. Automatic yara rule generation using biclustering. In *Proceedings of the 13th ACM Workshop on Artificial Intelli- gence and Security (AISec '20)*, pages 71–82, New York, NY, USA, November 2020. Association for Computing Machinery.

[13] Pilar Schummer, Alberto del Rio, Javier Serrano, David Jimenez, Guillermo Sánchez, and Álvaro Llorente. Machine learning-based network anomaly detection: Design, implementation, and evaluation. *AI*, 5(4):2967–2983, 2024.

[14] S. S. Soni and R. S. Soni. APIARY: An api-based automatic rule generator for yara to enhance malware detection. *Computers & Security*, 153:104397, 2025.